Lecture 1

1. Computer Generations

First (1944~1958): vacuum tube (MARK I, ENIAC, UNIVAC)
Second (1959~1963): transistor (IBM1401, IBM 1410 with 1402 card read/punch)
Third (1964~1970): IC (DEC PDP-1, IBM 360)
Fourth (1971~now): VLSI (very-large-scale integrated circuits) (Cray-1 supercomputer, Apple II, IBM PC, Notebook PC, Palmtop, Laptop, iPhone, iPad)

2. Computer System

Computer Hardware	Computer Software	
	System Software	Applications software
input	Communication with hardware	benefits or assists the user
processing	Resource management	
output	Facilitates communication among application programs	
storage		
communications		

3. Backward (Downward) Compatibility for new hardware(向下兼容性)

Most software written for computers with old hardware can be run on computers with newer hardware

4. VHDL

-Very high-speed integrated circuit Hardware Description Language
-A programming language to be used to specify both the structure and function of hardware circuit.
-Supports computer simulation as well as providing input to automatic layout packages which arranges the final circuits.

5. Hierarchy of the systems(计算机系统结构)

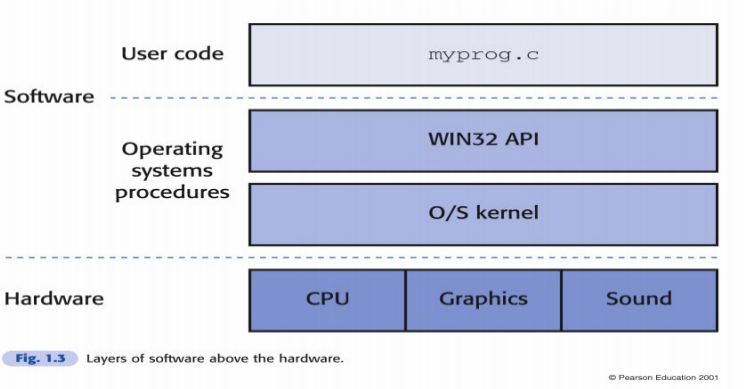


Fig. 1.3 Layers of software above the hardware.
© Pearson Education 2001

6. Advantages of the operation system

-Functionalities of the hardware systems can be brought out operating systems and thus offered to the user. (通过 OS 调取硬件的功能)
-The user's programs interact with hardware systems through the functionalities provided by operation system. (用户程序的功能由 OS 提供)

- Ease of programming.
- Protection for the system and for other users.
- Fairness and efficiency of using system resources.

7. Moore's Law

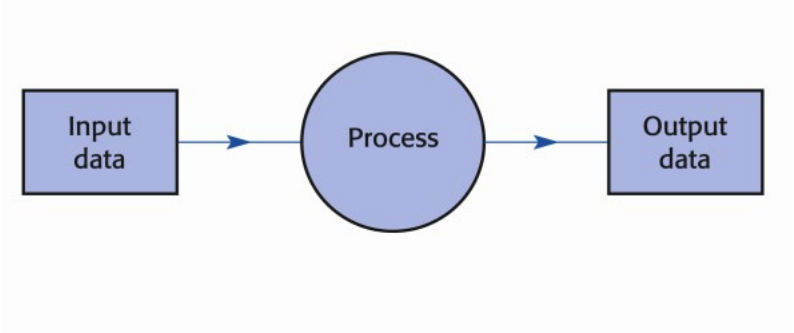
-A circuit designed 24 months ago can now be shrunk to fit into an area of half the size.
-It is sometimes quoted(缩短) as every 18 months.

8. Trends of Computing

- Scientific computing - computation
- Business computing – data
- Personal computing – interaction
- Pervasive computing – ubiquity(无处不在)
- Mobile computing – mobility

Lecture 2

1. Input-Process-Output Model



- A highly conceptualized computer model (高度概念化的计算机模型)
- The fundamental structure of the current generation of digital computers.
- The process is to be controlled by a special, custom-made program. (由特殊的定制程序执行)
- This was an essential scheme of the von Neumann model. (这是冯·诺伊曼模型的基本模型)

2. Components of the Computer System

Three components required for the implementation of Input-Process-Output and von Neumann model(s) 对于这两个模型必须的三大要素
-Hardware
-Software
-Data that is being manipulated

3. Hardware

1) Central Processing Unit (CPU)
   an active part which performs calculations and other operations.
2) Main memory(primary storage or working storage 主存储和工作存储)
   也可以叫做 RAM (random access memory)
   This holds data and programs for access by CPU.
3) Secondary storage
   - Long-term storage(长期保存)
   - Holds programs and data
   - Hard disk, CDs, DVD, etc.
4) Input devices
   keyboard, mouse, scanner, etc.
5) Output devices
   monitor, speaker, printer, etc.

4. Software

1) The hardware of a computer (e.g. CPU) can carry out only **very simple operations** like adding numbers (very quickly).
2) To make it perform useful tasks, these simple steps are combined in the form of **programs**, which are collectively known as **software**

5. Machine instructions

1) The CPU performs (执行) the execution(执行) of machine instructions.
2) Every CPU has its own instruction set (指令集) (100- 200 instructions, typically).
3) Although the instruction sets of different CPUs are similar, there is no standard instruction set. (对于指令集是没有统一标准的)

6. Machine Instruction Categories

1) Input-output
   - IN, OUT (Intel x86 and Pentium, but does not exist in some CPUs), …
2) Data transfer and manipulations
   - MOV, ADD, MUL, AND, OR, …
3) Transfer of program control
   - JMP, JC, …
4) Machine control
   - can halt processing(停止进程), reset the hardware(重新 set 硬件), INT, HLT…

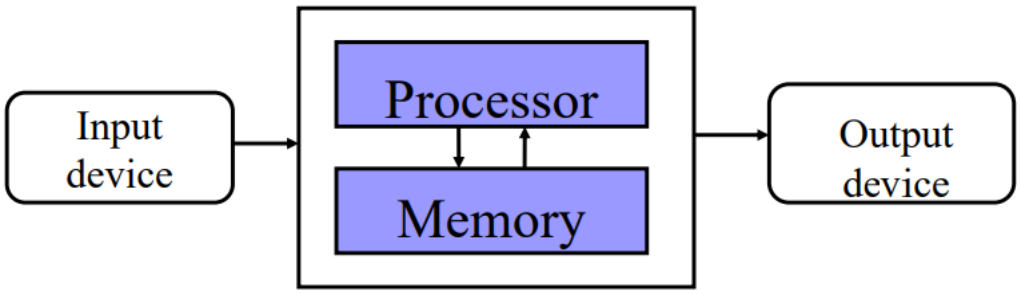
7. Machine Instructions and HLL

1) **H**igh **L**evel Programming **L**anguages (HLLs) are more suitable for programming than the languages of machine instructions.
2) The programs in HLL still have to be translated to the machine codes. (计算机只能处理机器码)

8. The Von Neumann Model

1) Ideal: The computer is a ***general-purpose*** machine controlled by an ***executable program.***
2) In this context:
   - A program is a list of instructions used to direct(指导) a task.
   - Both ***program*** and ***data*** are held in computer's memory (store) and both represented by binary codes.
   - The fact that **memory is re-writeable** makes a von Neumann machine especially powerful. (memory 可重复擦写是的冯诺依曼机器更强大)
   - A ***processor*** is an active part of the machine that executes the program instructions.

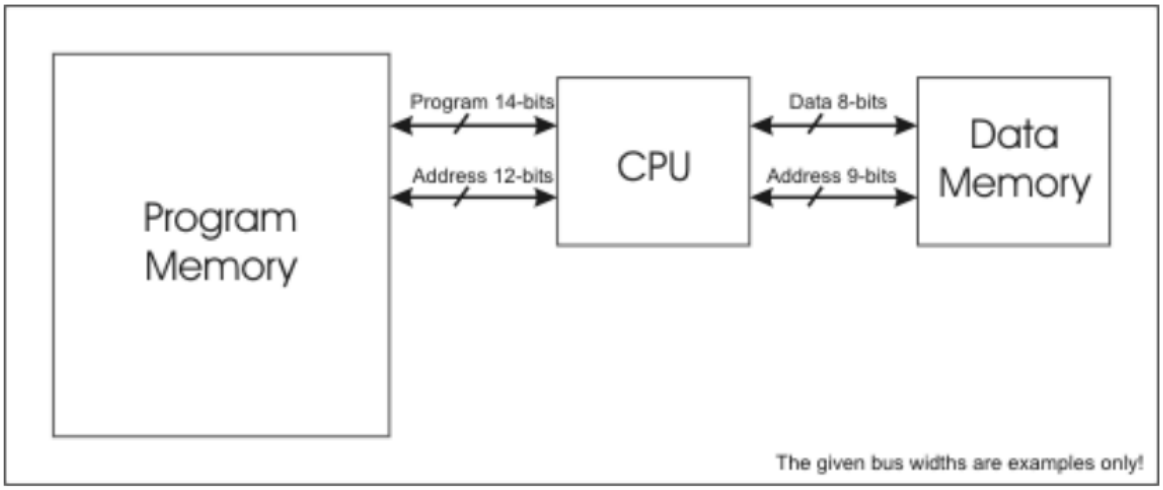
3) Figure of von Neumann Model:



4) Input device is for transmitting information from a user into the computer's memory.
5) Output device enables (使能够) a user to see results of the program being performed (执行).
6) Von Neumann bottleneck (冯·诺伊曼瓶颈)
   CPU is continuously forced to wait for vital data (and instructions) to be transferred to or from memory. (CPU 需要等待 data 和 instructions)
7) Potential problems
   - How could computers distinguish data from instructions since they are both represented by binary codes? (计算机如何区分数据与指令，因为它们都由二进制代码表示？)
   - A 16-bit instruction code could, in different circumstances, represent a number or two characters. (16 位指令代码在不同情况下可以表示一个或两个字符。)
8) Solution
   - Data and instructions are stored in memory.
   - CPU knows where to fetch (取得) program instructions.
   - Central Processing Unit (CPU) executes the program instructions.
   - Instructions and data have to be in a special coded form in order to be understood by CPU.

9. Harvard architecture

1) 特点
   - Separates data from programs
   - Requires different memories and access buses for programs and data. (需要不同的存储和访问总线的程序和数据。)
   - The intention is to increase transfer rates, improving throughput. (目的是提高传输速率，提高吞吐量。)
2) 原理图



10. The difference between 冯诺依曼机器&&Harvard Architecture

1) Von Neumann Architecture 的优点：
   Most memory (hard drives and RAM) can hold data as well as instructions
2) Harvard Architecture 的优点：
   Most desktop CPUs have an internal "instruction cache" feeding the control unit and a completely separate "data cache" (大多数桌面 CPU 具有内部"指令缓存"，为控制单元提供馈送和完全独立的"数据缓存")

Lecture 3

1. Semantic gap

Definition: The term expresses the enormous (巨大的) difference between the way human languages expressing ideas and actions and the way computer instructions representing data processing activities.

2. Translation(it is done by special programs,例如以下三个)

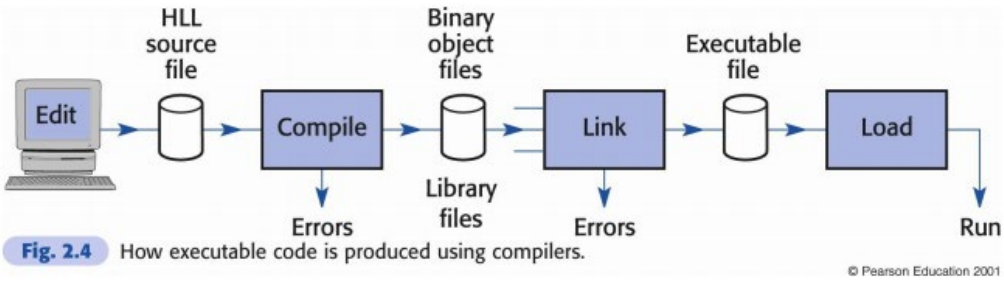


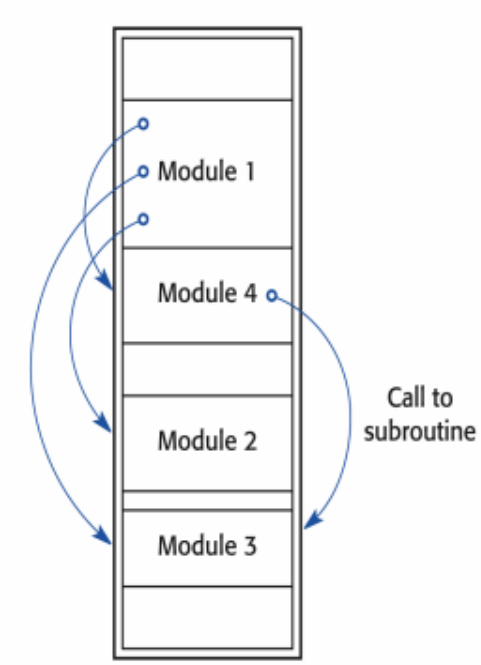
Fig. 2.4 How executable code is produced using compilers.

© Pearson Education 2001

1) Compilers (编译器): translating HLL instructions into machine code (sequence of instructions) before the code can be run on the machine.
2) Assemblers (汇编语言): translating mnemonic form of machine instructions (like MOV, ADD, etc) into their binary codes.
3) Interpreters (解释程序): translating HLL instructions into machine code on-the-fly (while the program is running).

3. Linking

1) Big programs usually are divided into several separate parts or modules. (大型程序通常分为几个单独的部分或模块)
2) Each module has to be designed, coded and compiled。
3) There are frequent occasions when code in one module needs to reference data or subroutines in another module. (在很多情况下，一个模块中的代码需要引用另一模块中的数据或子例程)



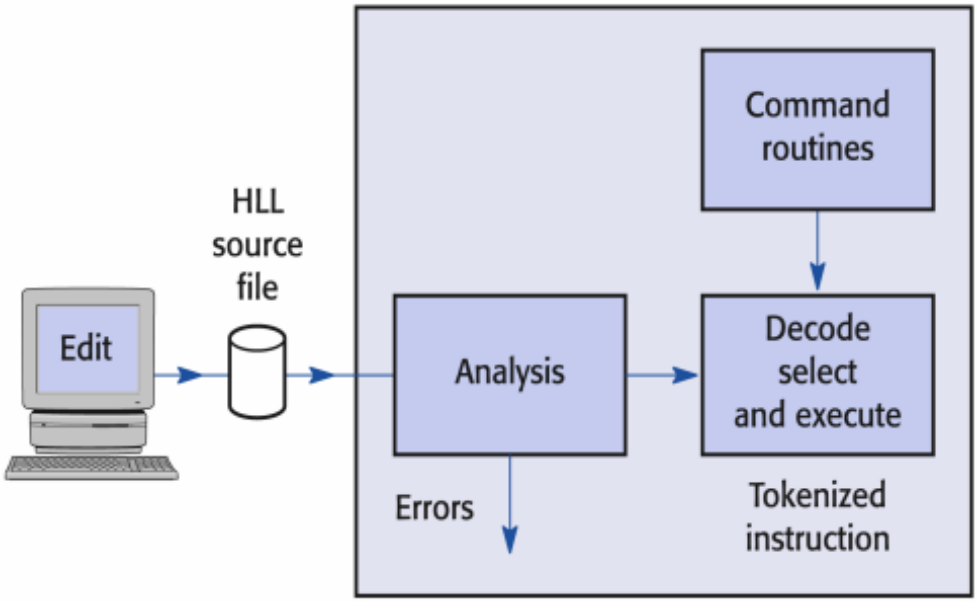
4) A compiler can translate a module into binary codes, but it cannot resolve those references to other modules. (编译器可以将一个模块转换成二进制代码，但不能解析对其他模块的引用)
5) Those external references remain symbolic after the compilation, until the linker gets to work. (编译之后，这些外部 reference 仍然是符号的(抽象的)，直到 Linker 开始工作)
   - The linker is to join together all the binary parts.
   - The linker will report errors if it cannot find the module or code referred to by those external references. (如果链接器找不到外部引用所引用的模块或代码，则会报告错误)

4. Library files

1) Translated object code.
2) Provide many functions for programmers, but are only usable if linked into your code.
3) In Unix:
   - Directories/lib and user/lib/.
4) In Windows:
   - DDL files.

5. Interpreters (解释程序)

1) Alternative way of running HLL programs



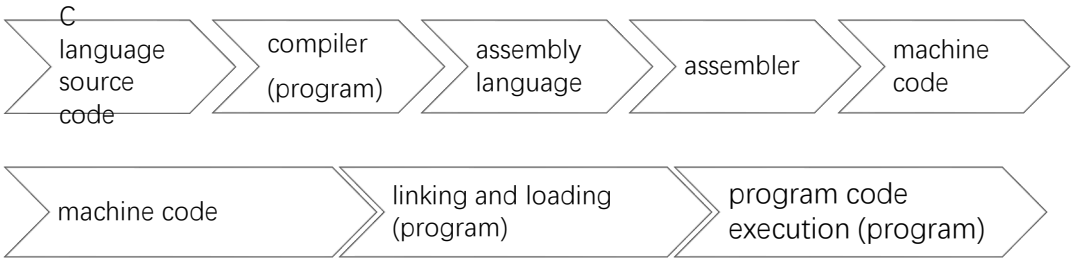
2) Such as used with BASIC and Java.
3) Instructions are converted (转化) into an intermediate (中间的) form, consisting of tokens (符号).
   - In Java, tokens such as: static, Boolean, file, string, void, return
4) Tokens are then passed to the decoder (解码器), which selects appropriate routines for execution (执行).

6. Compilers 和 Interpreters 的总结(两者的区别就在于 compiler 是先全转成二进制再执行，而 interpreters 是转一行 instruction 执行一行)

1) Compilers
   - Take a program and translate it as a whole into machine code.
   - The processes of translation and execution are separate.
2) Interpreters
   - Take an instruction, one at a time, translate and execute it.
   - The processes of translation and execution are interlaced (交错的).

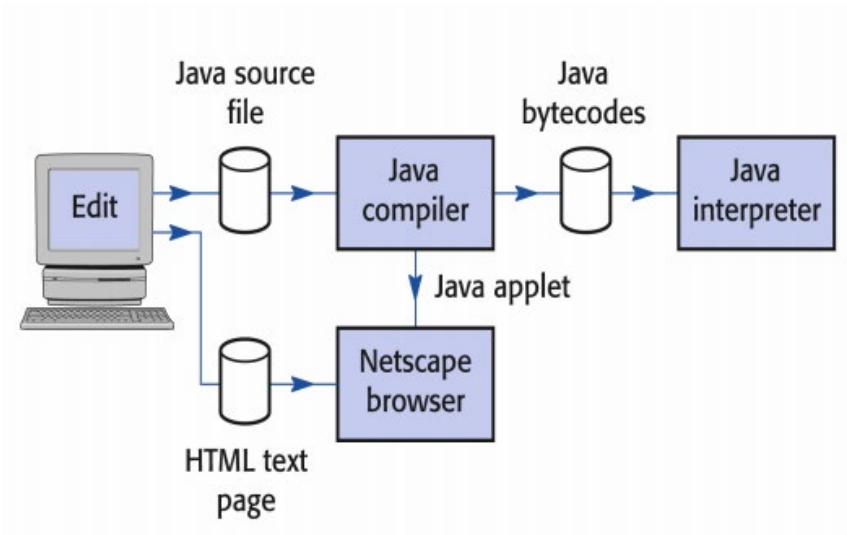
7. 程序运行的两个例子(C 和 Java)

1) C:



2) Java：





8. Interpreters vs. Compilers

1) Execution of compiled code is much faster than execution of interpreted code. (But this disadvantage of interpretation is not a big problem for most applications.)对于大部分程序来说这个问题不大
2) Interpreters are more suitable for rapid prototyping(初始设计) and for other situations when a program is frequently modified.(解释器更适合快速原型设计，以及程序经常被修改时的其他情况)
   - Interpreters are more accurate(准确) in terms of error reporting.
   - Interpretation can provide uniform execution environment across several diverse computers. (Portable)(解释器可以在多台不同的计算机上提供统一的执行环境（便携的）)

9. Interpreters as Virtual Machines

1) Interpreters are somewhat similar to the computer hardware (CPU)(解释器与计算机硬件(CPU)有些相似)
   – take one instruction at a time and execute it.

2) Because of that sometimes they are referred to as a virtual machine(因此，有时它们被称为虚拟机)
-Example: JVM, Java Virtual Machine

10. Code sharing and reuse

1) How to reuse existing proven software when developing new systems?
   - Source-level subroutines(子程序) and macro(宏) libraries.
   - Pre-translated re-locatable binary libraries. (预翻译可重新存取的二进制库)
   - Dynamic(动态的) libraries and dynamic linking
2) How code can be share

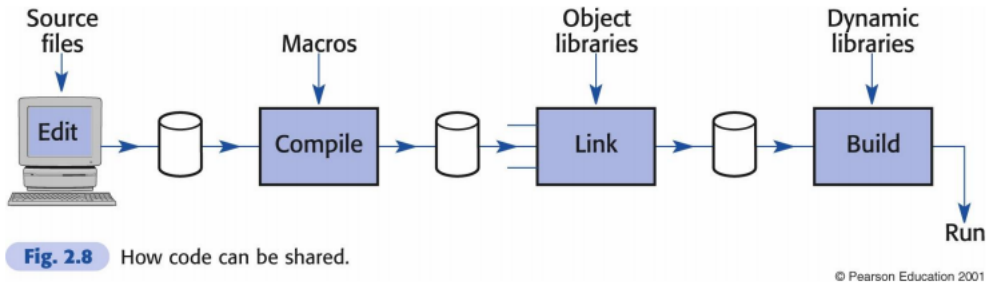


Fig. 2.8 How code can be shared.

© Pearson Education 2001

11. Macro (宏命令)

Macro example

```
macro Print_strings (s1, s2)
printf("%s", s1);
printf(" %s", s2);
End macro

main()
{
string a, b;
 …
Print_strings (a, b);
…
Print_strings("Hello","world");
…
Print_strings("Game","over");
…
}
```

Expanded macro

```
main()
{
string a, b;
 …
Print_strings (a, b);
…
Print_strings("Hello","world");
…
Print_strings("Game","over");
…
}

main()
{
string a, b;
 …
printf ("%s", a);
printf ( "%s", b);
…
printf ("%s", "Hello");
printf (" %s", "world");
…
printf ("%s", "Game");
printf (" %s", "over");
…
}
```

12. Source-level subroutines(源代码级子程序) and macro libraries

1) Intention(目的/意图):
   - Take copies of the library routines.
   - Edit them into your new code.
   - Translate the whole together.
2) Disadvantages:
   - Who owned the code?
   - Who should maintain(维护) it?

13. Pre-translated relocatable binary libraries(预翻译的可重定位二进制库)

1) Intention:
   - Libraries are pre-translated into relocatable binary code.
   - Can be linked into your new code, but not altered.
2) Acceptance(接受度):
   - Successful, and still essential for all software development undertaken today. (成功，对于今天进行的所有软件开发仍然至关重要)
3) Disadvantage:
   - Each program is to have a private copy of the subroutines(子程序), wasting valuable memory space, and swapping time, in a multitasking(多(重)任务处理) system.
4) Relocatable---------- adj. [计] 浮动的，[计] 可再定址的

14. Dynamic(动态的) libraries and dynamic linking

1) Intention:
   - Load a program which uses “public” routines(程序) already loaded into memory.
   - The memory-resident(存储驻留) libraries are mapped(映射), through the memory management system to control access and avoid multiple code copies.(通过内存管理系统映射内存驻留库，以控制访问并避免多个代码副本)
2) Acceptance
   - Successful through Microsoft’s ActiveX standard.(通过微软的 ActiveX 标准)

Lecture 4

1. Data, Information and Knowledge

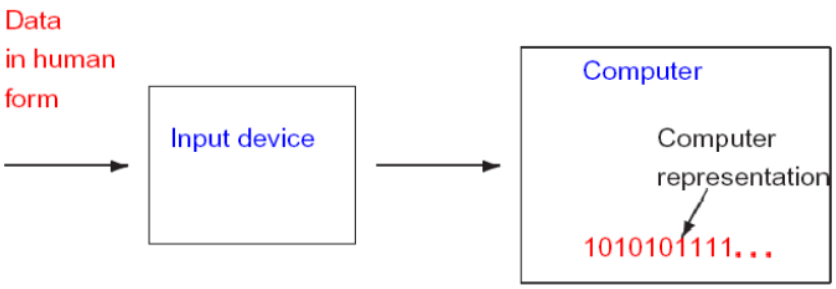
1) Data – raw facts, figures, measurements, …
2) Information
   - data organized into useful representation
   - 1.000, 1.000, 2.000, 3.000, 5.000,
3) Knowledge
   - application of reasoned analysis of information
   - 'data are in increasing order', 'data can be derived based on Fibonacci sequencing', etc

2. Measurement of Information

1) 1948~~~ Claude Shannon~~~the first truly mathematical theory of **entropy(**熵**)**.
2) His ideas created two main lines of development: **information theory** and **coding theory.**
3) Entropy of an event X is related to p(X)

3. Data codes – numeric and character

1) **Binary number system(二进制系统)**------a system of choice, both for data storage and for internal processing of operations.
   (Based on the two-state, ON/OFF technology)
2) **Decimal number system(十进制系统)** ------ 人类使用
3) Diagram:



4. Alphanumeric data(字母数字数据)(大部分的数据最初是以 letter in

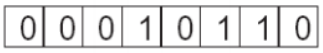
alphanumeric, numbers 和 punctuation(标点))

1) They are represented in computers by binary numbers.

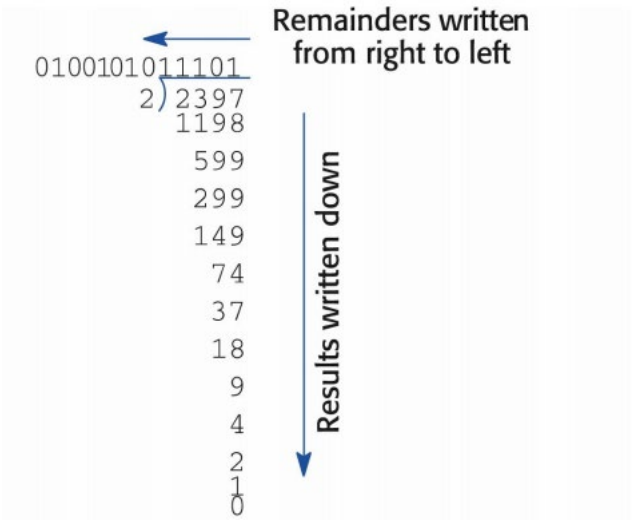
5. Bit

1) **A bit is the most basic unit of information** possible: it contains the information necessary to distinguish two alternatives (1 or 0, YES or NOT, etc.).
2) We think of these alternatives as being numbers 0 and 1.
3) Two alternatives are represented by two-state elements (memory cell is charged, or not)。

6. Binary number system

1) A sequence of bits (binary digits) represents a number in the binary notation
2) Example: the sequence 

represents the number 22: $10110_2 = 22_{10}$.
3)



7. Binary vs. decimal notation

1) Decimal notation is more compact(紧凑的) than binary.
2) Decimal is more convenient(方便) for humans.
3) Binary is more “convenient” for computers due to two-state, ON/OFF technology employed.

8. Hexadecimal notation

1) Why hexadecimal, or base 16, number notation?
   – Convenient shorthand for binary numbers.
   – Every hexadecimal digit exactly represents 4 binary bits.

$0 = 0000_2$	$8 = 1000_2$
$1 = 0001_2$	$9 = 1001_2$
$2 = 0010_2$	$A = 1010_2$
$3 = 0011_2$	$B = 1011_2$
$4 = 0100_2$	$C = 1100_2$
$5 = 0101_2$	$D = 1101_2$
$6 = 0110_2$	$E = 1110_2$
$7 = 0111_2$	$F = 1111_2$

2) Translation from binary to hexadecimal:
   - First breaking it into 4 digits groups (from right to left)
   - Translating each group into one hexadecimal digit:
     $10011110_2 = 9E_{16}$
3) Translation from hexadecimal to binary:
   - Translate every hexadecimal digit into the 4-digit binary pattern.
     $AA3E_{16} = 1010101000111110_2$

9. How data are represented in a computer – Summary

ASCII Table (7-bit)
(ASCII = American Standard Code for Information Interchange)

Decimal	Octal	Hex	Binary	Value
048	060	030	00110000	**0**
049	061	031	00110001	**1**
050	062	032	00110010	**2**
056	070	038	00111000	**8**
057	071	039	00111001	**9**
065	101	041	01000001	**A**
066	102	042	01000010	**B**
097	141	061	01100001	**a**
098	142	062	01100010	**b**

Lecture 5

1. Alphanumeric characters(字母数字字符组)

Data that computers deal with usually comprise(包含):

- Letters, such as 'A', 'B', etc.
- Numbers, such as '1', '9', etc.
- Other characters, such as punctuation marks.

2. Binary codes of alphanumeric characters

1) The need for coding.
   由于所有数据都由计算机中的二进制序列表示，因此问题就涉及到如何表示它们。
2) The choice of code may be arbitrary. (代码的选择可能是任意的)
3) But what matters is consistency, i.e. **standards** are needed.
   (但是，重要的是一致性，即 需要标准。)

3. Alphanumeric codes

1) **ASCII** code (**A**merican **S**tandard **C**ode for **I**nformation **i**nterchange (**7-bit** code) and its extensions (8-bit codes) (well-established).
2) **EBCDIC** code (**E**xtended **B**inary **C**oded **D**ecimal **I**nterchange **C**ode) **8-bit** code. (IBM mainframe computers)
3) **Unicode**. Recent **16-bit** standard. (Up to 216 characters can be encoded)

4. ASCII code table

1) It is 7-bit code, so the code of any character can be represented as a two-digit hexadecimal number (row's and column's numbers are the first and second digits, respectively)(这是 7bit，因此任何字符的代码都可以表示为两位数的十六进制数字（行号和列号分别是第一位和第二位)
2) 3 bits are represented by the first hexadecimal digit and the next 4-bits are represented by the second hexadecimal digit. (第一个十六进制数字表示 3 位，第二个十六进制数字表示接下来的 4 位)
3) Only half of possible byte (8-bits) patterns is used.
4) The second half is used in the 8-bit extension(s) to represent the codes of additional symbols, line shapes, foreign letters, etc. (后半部分用于 8 位扩展（s）表示附加符号、线形状、外文字母等的代码)
5) The order of the letter codes is compatible with the alphabetical order of the letters. (字母代码的顺序与字母的字母顺序兼容。)

6) The table is divided into two classes of codes:
   - Printing characters
     --- produce output on the screen or on a printer.
   - Control characters
     --- To control the position of output on the screen or paper (e.g. 'HT').
     --- To cause some action to occur (e.g. 'BEL').
     --- To communicate status between the computer and an I/O device (e.g. 'Control-C' combination).
7) Limitations of ASCII code (for well-established 8 bit ASCII codes)
   - Too limited for the display requirements of modern Windows-based word-processors.
   - The requirement of global software market for handling **international** character sets.
     (全球软件市场对处理国际字符集的要求)

5. Unicode(万国码)

1) Even 8-bit extensions of ASCII code table is capable(能够) to code only up to 256 characters.
2) Unicode Standard (1991) is an 16-bit international encoding system for information interchange.
   --- Code values are available for more than 65,000 characters.
3) Standard includes the European alphabetic scripts, Middle Eastern right-to-left scripts, and scripts of Asia, Africa and America, ideographic characters of China, Japan, etc.
4) Standard includes punctuation marks, mathematical symbols, geometrical shapes, etc.
5) Is it OK to use Unicode in Java programming?
   ---In a Java program, if you want to show some Chinese characters, you have to use their Unicode representations.

6. Representation of numbers

1) Any representation of numbers capable(能够) of supporting arithmetic manipulation(算数运算) has to deal with both **integers** and **real values**, **positive** and **negative**.
   - Integers are whole numbers, with no fraction part (小数部分).
   - Real numbers extend beyond the decimal point.

7. Representation of integers

1) Generally 4 bytes, i.e. 32 bits.
2) Two's complement as a method of representing and manipulating negative integers.

-128	64	32	16	8	4	2	1

$00000101_2 = 5_{10}$ $\quad$ $11111011_2 = -5_{10}$

先去掉正数的头，然后取补码，然后+1，最后再给它安个"1"的头

8. Representation of real numbers

1) IEEE 754 standard.
   - The most widely-used standard for floating-point computation.
   - defines formats for representing floating-point numbers, special values, and a set of floating-point operations that operate on these values. (定义表示浮点数、特殊值和一组基于这些值操作的浮点操作的格式)

9. Declaration of variables in programs(程序中变量的声明)

1) What happens when you declare variables in a program?
   - You are telling the compiler to reserve the correct amount of memory space to hold the variable.
     (您告诉编译器保留正确的内存空间量来保存变量。)
   - You are also telling the compiler what encoding/decoding/representation scheme to be used.
     (您还告诉编译器要使用什么编码/解码/表示方案。)

`char letter`	1 byte
`short count`	2 bytes
`long world_population`	4 bytes
`long double world_weight`	8 bytes

Lecture 6

1. Operating systems: examples

Operating systems have undergone more than 40 years of evolution.
(操作系统经历了 40 多年的发展)

2. OS---Functions

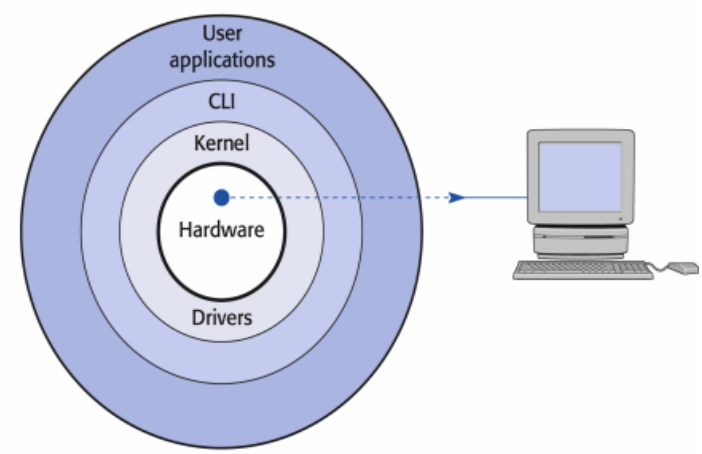
1) Functions of OS - to interpret(解释) and carry out(执行) commands issued by:
   - Users of the computer
   - Application programs running on the computer.

3. OS---Purposes(目的)

1) Two fundamental purposes:
   - Management(管理):
     - To control and operate hardware in an efficient way.
   - Provide functionalities(提供功能):
     - To allow the user **efficient access** to the facilities of the machine. (允许用户高效访问机器的设施)
     - To allow the user **fair access** to the facilities of the machine. (允许用户公平访问机器的设施)
     - To allow the user **protected access** to the facilities of the machine. (允许用户对计算机设施进行受保护的访问)
     - **Interaction** with the user. (与用户的交互)

4. OS---Purposes – Onion ring model

1) To realise(实现) the purposes of an operating system, the software system is often arranged in multiple layers.(通常是多层排列的)
2) Gatekeeper(看门人)--- We use a computer through the operating system.
3) Butler(管家)--- It is often the operating system that governs the overall efficiency of a computer. (操作系统通常决定着计算机的整体效率)
4) Diagram of the Onion ring model:



5) **Core of operating system**: dealing directly with the hardware.
- **Kernel**: device drivers, memory allocator…(内核:设备驱动程序,内存分配器...)

6) **CLI**: provide user accessibilities to the system.
(提供用户对系统的访问权限)

5. Modern operating Systems

Many modern operating systems also:

- Allow for the simultaneous(同时发生的) processing of multiple programs.(允许同时处理多个程序)
  - DOS:
    ---To run a second program you have to wait the current one finishes.(单任务处理)
    ---Background spooling provides minimal degree of concurrency(后台多任务缓冲处理器提供极小的并发性)
  - Windows:
    Programs can run simultaneously, if they are not too resource consuming. (Same with Unix and Linux.) (Multi-tasking)(多任务处理)
- Support for multiple users. (Multi-user)(支持多用户)

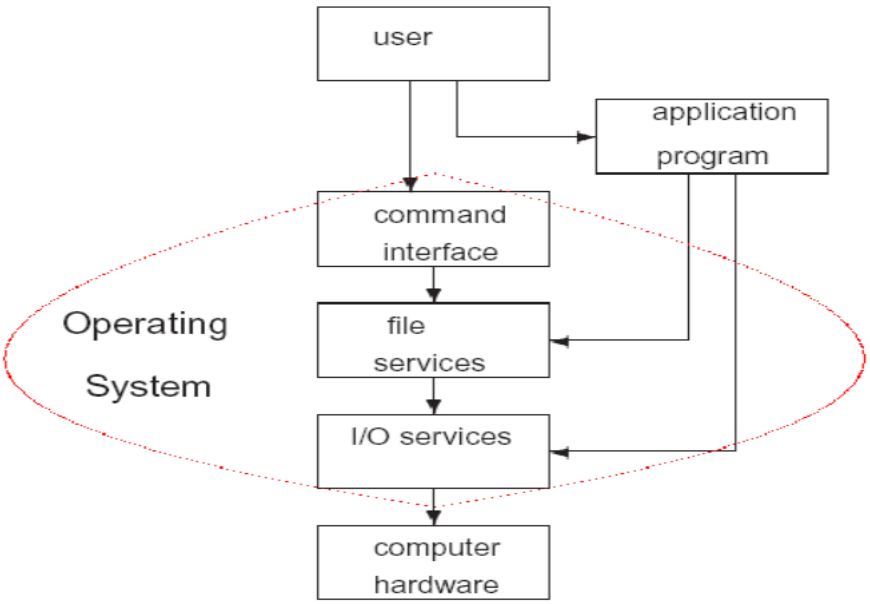
6. Interaction with operating system

1) CLI (command line interpreter)(命令行解释器)
   - DOS: type a command in a command line.
     ---在命令行中键入命令
   - Unix/Linux: shell scripts (sequences of instructions).
     --- 外壳脚本（指令序列）。
   - Windows/Mac OS X: click with mouse on icons(图标).
     ---用鼠标单击图标
2) Function calls from within user programs (API).
(来自用户程序内的函数调用)

7. Complexity of OS operation（操作系统的复杂性）

1) To accept a command from a user, OS must:
   - Accept the input keystrokes from the keyboard.
   - Interpret this keystrokes as a command.
   - Determine the location of the file with a program for this command
   - Read the appropriate blocks of file from (secondary storage) device into main memory.
   - Set up context for the program to be executed.
   - Transfer control to the program being executed.
   - Resume control when the program is finished.
2) If multiple programs are executing simultaneously, (如果多个程序同时执行), OS must include also:
   - Program to allocate(分配) memory and other resources to each program. (memory manager)---内存管理
   - Program to allocate CPU time to each program (scheduler: more details later on).---调度
   - Program to maintain integrity of each program. (security kernel) 维护每个程序完整性的计划（安全内核）
   - ……

8. Simplified picture of OS services



9. Computer Networks（影响+定义）

1) Perhaps the most far-reaching changes(最深远的变化) ever produced to von Neumann's original blueprint.
2) OS usually provides access to network facilities. (via networking API, e.g., socket interface)
   (操作系统通常提供对网络设施的访问(通过网络 API ,例如,套接字接口))
3) Computer network is an interconnected(互相连接的) collection of autonomous computers to facilitate fast information exchange.
   计算机网络是自动计算机的互连集合，便于快速交换信息。

10. Why computer networks?

1) To increase computing power.
   - Distributed computing projects(分布式计算项目): 例如 SETI@home, Folding@home.
2) To share valuable resources (printers, large disks facilities, programs, data bases, etc.).(分享资源)
   - The central printing system at XJTLU.
   - The central filing system at XJTLU.
3) To organize convenient interactions of users working at their own computers, often from different locations.
   组织用户在自己的电脑上(通常是在不同地点)进行方便的交互
   - Think about online games!
4) Information at your fingertips…（信息触手可及...）

11. Connectivity

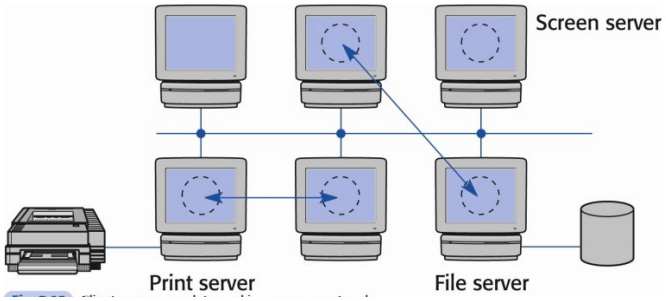
1) Connecting devices for communication 连接设备进行通信
   (voice, data, multimedia)
   - foundation for information age(信息时代基础)
2) E-mail
   - send and receive messages over a local area network or a large network, including the Internet
     (通过时位网络或大型网络（包括 Internet）发送和接收消息)
3) Telecommuting
   - Working at home or on the road
   - Communicating with the office through phone, fax, and/or computer
4) On-line shopping, E- or M-commerce

12. Browsers and the World Wide Web(浏览器和万维网)

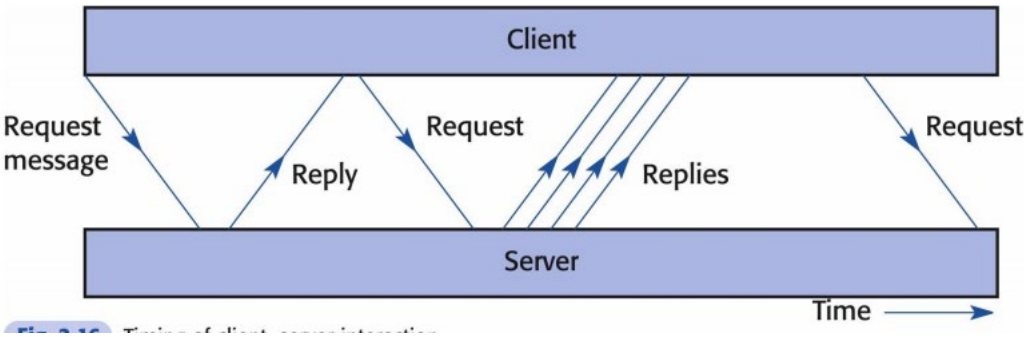
- Microsoft Internet Explorer and Netscape Navigator
  --- two dominant Web browsers

13. Client-server computing(客户端-服务器计算)

1) Client: The originator of a request. (发起者)
2) Server: The supplier of the service. (服务提供者)



14. Client-server interaction



1) Client starts the interaction by sending a request message to the server.
   客户端通过向服务器发送请求消息来启动交互。
2) Server responds by sending replies back…
   服务器通过发送回复来响应...
3) Look at the status bar of your web browser when opening a web page from a remote site.
   从远程网站打开网页时，查看 Web 浏览器的状态栏。

Lecture 7

1. The principal components of a computer(计算机的主要组件)

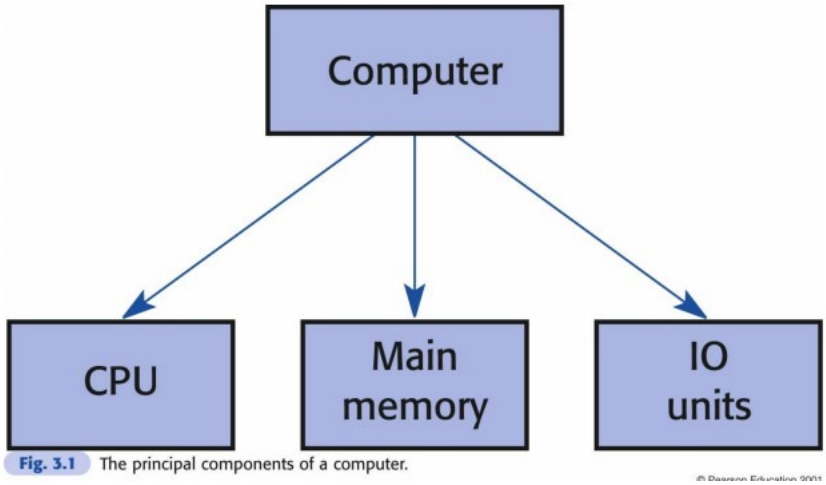
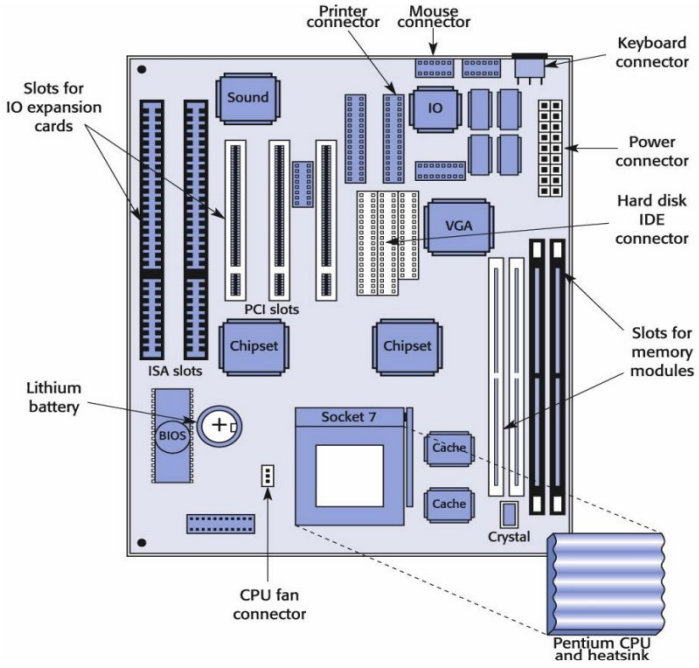


Fig. 3.1 The principal components of a computer.

1) These are the minimum set of components for a working digital computer. (这些是工作数字计算机的最低组件集)
2) A PC motherboard often appears much more complicated. (pc 主板更加复杂)

2. Motherboard

1) Three principal subsystems:
   - CPU
   - Main memory
   - Input-output units
2) Each of these is often made up of many components



3. Processor **处理器**

1) **arithmetic/logic unit**(ALU)(算术/逻辑单元)
2) **control unit**: part of a CPU responsible for performing the machine cycle - fetch, decode, execute, store
(CPU 中一部分负责执行机器周期——读取、解码、执行、存储)

4. Registers**(寄存器)**

1) **Program counter** (PC): contains the address of the next instruction to execute. (包含下一条要执行的指令的地址)
2) **Instruction register** (IR): part of a CPU control unit that stores an instruction. (CPU 控制单元的一部分，用来存储一条指令)

5. Coprocessors(辅助处理器): Assistants(助手) to the CPU

- Microprocessors(微处理器) performing specialized functions that CPU cannot perform or cannot perform as well and as quickly
(微处理器执行的特殊功能是 CPU 无法执行或无法快速执行的)
  - math
  - graphics

6. Boards and Chips

1) Circuit boards
2) Use aluminum or copper to conduct electronic messages
(使用铝或铜来传递电子信息)
3) Chips of silicon(硅芯片)
4) Semiconductor(半导体)

7. Buses

1) On the motherboard, all the components are interconnected by buses ("signal highways").(主板上所有原件都由 bus 连接)
2) A bus is a bundle of conductors(导体), wires(导线), or tracks(轨迹).
3) Typically, there are **address(地址)**, **data(数据)** and **control(控制) buses**, each including several signal lines.
   - Intel 8086: 20 shared address/data lines, and a further 17 lines for control.
   - Intel Pentium: data bus 64 lines, and the address bus 32 lines.
4) Each hardware unit is connected to all these buses.
(每个硬件单元都连接到所有这些总线)
   - A simple way of building up complex systems in which each unit can communicate with each other.

(一种建立复杂系统的简单方法，其中每个单元可以相互通信)

- Little disruption when plugging in new units and swapping out failed units. (当插入新单元和交换出故障单元时，有微小的中断)

8. System interconnections(系统互连)

1) The connected devices can have access to any signal line they require (连接的设备可以访问它们需要的任何信号线)
2) Bus interconnection is often represented in diagrams as a wide pathway, rather than showing the individual wires.
(总线互连通常在图中表示为一个宽通道，而不是显示单个导线)

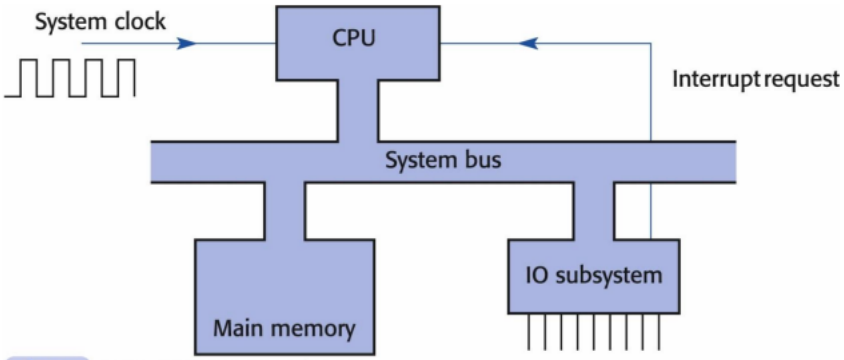


Fig. 3.3 System interconnection schematic.
© PearsonEducation 2001

9. Bus vs. point-to-point connections

1) An alternative scheme: point-to-point interconnection.
   - The number of pathways needed to link every possible pair of n units: $\frac{n(n-1)}{2}$.
   - Each pathway will still require a full-width data highway.
   (每个通道仍然需要一个全宽的数据高速公路)
   - The result will be a huge number of wires.

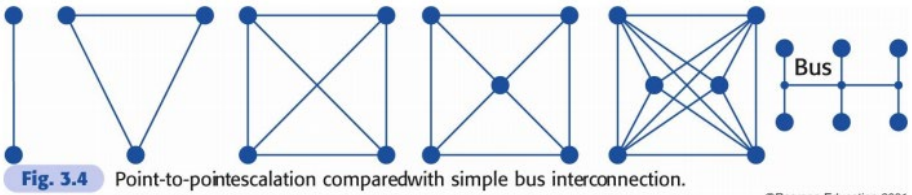


Fig. 3.4 Point-to-pointescalation comparedwith simple bus interconnection.
©Pearson Education 2001

2) The number of wires required for a bus is much smaller than that for point-to-point connections.(bus 需要导线数少)
3) A bus can only transfer one item of data at a time, like a railway line.
（bus 单次只可以传输 one term of data）
   - Leads to a limit on the performance, termed the **Bus Bottleneck.**
   （导致性能限制，即总线瓶颈。）
   - It cannot be solved by simply increasing the speed of a processor.
   （无法仅仅通过提高处理器的速度来解决此问题）

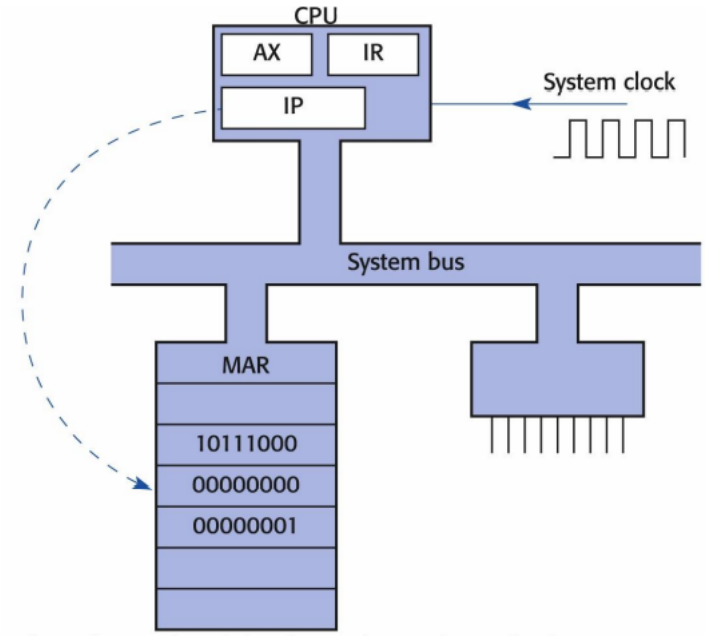
10. Two parts of CPU

- ALU(Arithmetic and Logic Unit)
- Control unit

11. Registers

1) CPU registers: small block of fast memory(小块快速存储器).
   - Temporarily store for data and address variables(暂时储存)
2) Some CPU registers:
   - **Instruction Pointer (IP)** or Program Counter (PC).
     (Stores the address of the *next instruction*)
   - **Accumulator** (AX, EAX in Pentium)
     (General purpose data register.)通用数据寄存器
   - **Instruction Register (IR)** 指令寄存器
     (Stores *the instruction that is being executed*.)
3) **Memory address register (MAR)**.
   - Temporarily holds address of the memory location during a bus transfer.(在总线传输期间临时保存内存位置的地址)
4) **MBR**

----------我是和谐的分割线（下图为 registers）----------



12. Computers speak this language: Instruction Set

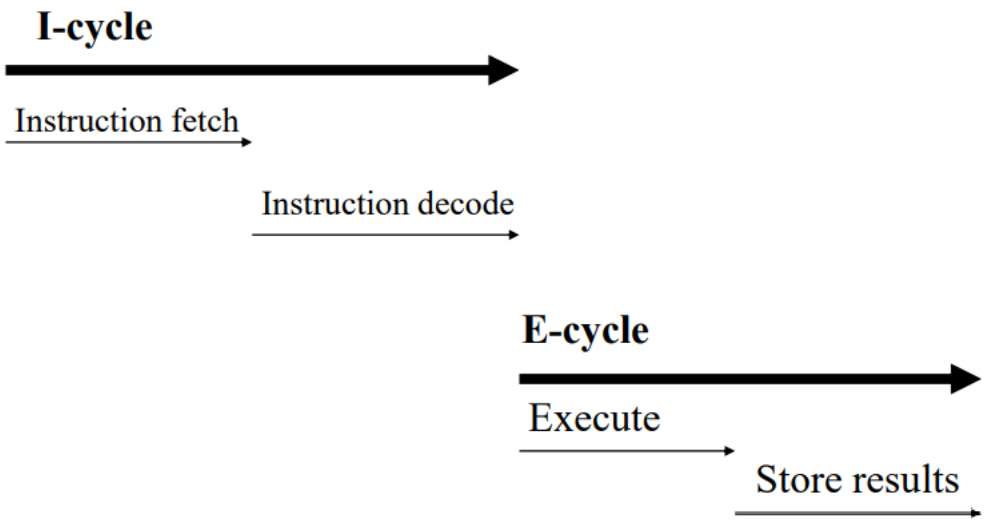
1) The collection of machine language instructions that a particular processor understands,
   (一个特定处理器所能理解的机器语言指令的集合)
2) Machine language instructions

- instructions for a specific CPU
  --- designed to be executed by a computer without being translated
  --- Also called **machine code**
  --- Operations(运算) like: ADD, SUB, INC, DEC, etc.

13. How instructions are executed?

1) The basic operation, known as the fetch-execute cycle or machine cycle
   - The sequence whereby each instruction of the program is executed(程序中每条指令执行的序列):
     --- Read from the memory.(从内存中读取)
     --- Decoded.(解码)
     --- Executed.(运行)

14. Machine Cycle（机器周期）



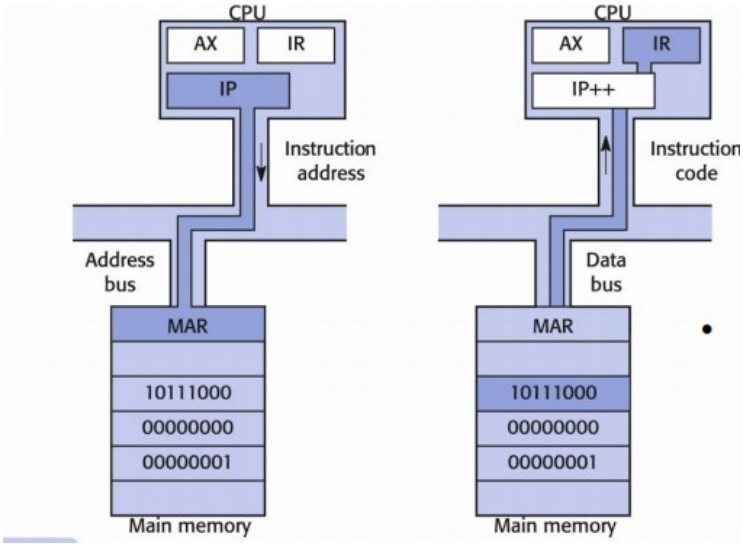
1) Fetch the instruction from memory
   - *This step brings the **instruction into the instruction register***, a circuit that holds the instruction so that it can be decoded and executed
2) Decode the instruction
3) [Read the effective address from memory if the instruction has an indirect address]
4) Execute the instruction
5) [Store the results]

总的看来就是抓取 instruction，对 instruction 执行 decode（解码），然后执行 instruction，最后储存数据。

----------对于机器周期中步骤的详细解释----------

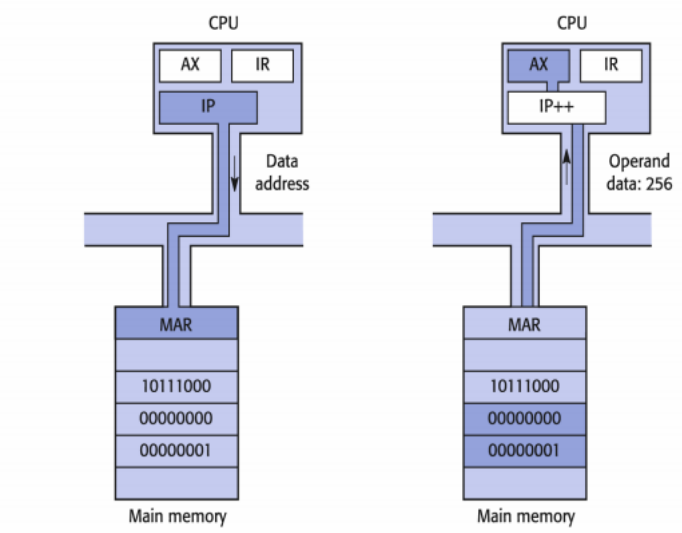
15. The *fetch phase* of the cycle (in principle)

1) The address in IP register is copied onto the address bus and further to MAR register.
   (IP 寄存器中的地址被复制到地址总线上,并进一步复制到 MAR 寄存器中)
2) IP is incremented ready for the next cycle. IP now points to the next location in the program memory.
   (IP 增量为下一个周期做好准备。IP 现在指向程序内存中的下一个位置)
3) Memory selects location and copies the content onto the data bus.
   (内存选择位置并将内容复制到数据总线上)
4) CPU copies the instruction code from the data bus into IR.
   (CPU 将指令代码从数据总线复制到 IR 中)
5) Decoding of the instruction starts.
   (指令开始解码)



16. The *execution phase* of the cycle

1) Execute phase depends on the type of instruction.
2) Example:
   the execution of MOV AX,256 instruction includes:
   - IP is copied to address bus and latched into memory.
     (IP 被复制到地址总线并锁存到内存中)
   - IP is incremented.
     (IP 增加)
   - The value selected in memory is copied onto the data bus
     (将 memory 中选择的值复制到数据总线上)
   - CPU copies the value from the data bus into AX.
     (CPU 将值从数据总线复制到 AX)



---

17. CISC("sisk")----(instruction may be several words long, requiring several fetches)

- **complex instruction set(**复杂指令集**)**
- most mainframes and PCs(大多数大型机和个人电脑)

18. RISC ("risk")----(instruction usually consists of a single word)

- **reduced instruction set(**精简指令集**)**
- cheaper and faster
- shift some work to software(把一些工作转移到软件上)

19. RISC is faster because …

- The vacated area of the chip can be used to accelerate the performance of more commonly used instructions, rather than compensating for those rarely used instructions
  (芯片的空闲区域可用于加速执行更常用的指令，而不是补偿那些很少使用的指令)
- Easier to optimize the design
  (更容易优化设计)
- Simplifies translation from high-level languages into the smaller instruction set that the hardware understands, resulting in more efficient programs
  (简化了从高级语言到硬件可理解的更小指令集的转换，从而产生更有效的程序)

20. Output Hardware

1) Hardcopy output
   - Graphics

- Letter

2) Softcopy output
   - Video
   - Audio (music from MP3 for example)

21. Screen Clarity(屏幕清晰度)

- Standard screen resolutions
  – 640 x 480
  – 800 x 600
  – 1024 x 768
  – 1280 x 1024
  – 1600 x 1200

22. Communications Hardware(通信硬件)

- Facilitate networks(促进网络)
  – modems (调制解调器)
  – hubs and other components of a network(集线器和网络的其他组成部分)

23. Ports: connecting peripherals to the computers（端口：将外围设备连接到计算机）

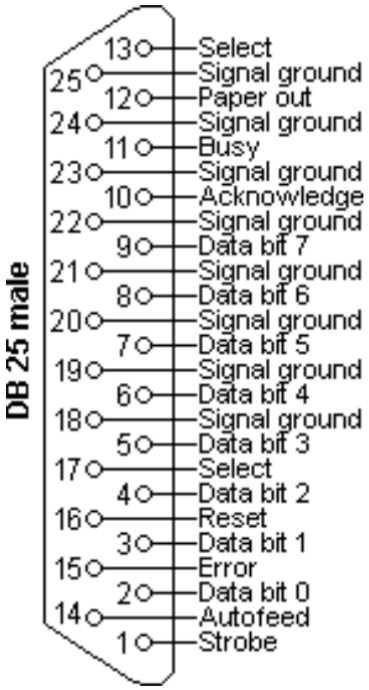
1) Parallel port (IEEE 1284)（并行端口）
   – printers, some scanners
2) Serial port (RS-232)（串口）
   – modems(调制解调器), scanners, mice(鼠标)
3) These now being replaced by …

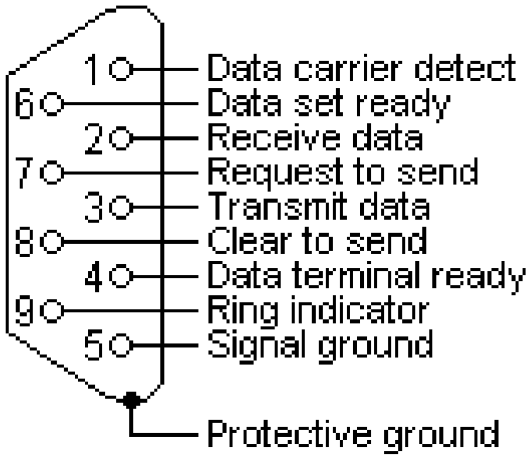
24. USB (Universal Serial Bus)（通用串行总线）

1) industry standard developed in the mid-1990s that defines the cables, connectors and protocols used for connection, communication and power supply between computers and electronic devices.
   (20 世纪 90 年代中期制定的行业标准，定义了用于计算机和电子设备之间的连接、通信和电源供应的电缆、连接器和协议)
2) standardized the connection of computer peripherals, such as keyboards, pointing devices, digital cameras, printers, portable media players, disk drives and network adapters to PCs.
   (标准化计算机外设的连接，如键盘、指向设备、数码相机、打印机、便携式媒体播放器、磁盘驱动器和网络适配器到个人电脑)
3) replaced earlier interfaces, such as serial and parallel ports, as well as separate power chargers for portable devices.

(更换了以前的接口，如串口和并口，以及采用便携式设备的独立电源充电器)

25. Connectors





26. Power supply

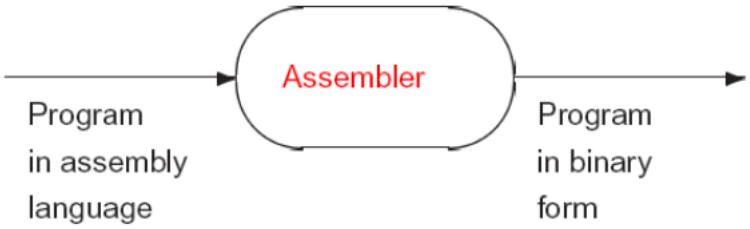
- protected by power surge protector（电源浪涌保护器保护）
- uninterrupted power supply unit (UPS)（不间断电源装置（UPS））

Lecture 8

1. Mnemonic form(助记符) and assembly language

1) In order to study more closely the way in which programs are performed, we will write some simple programs for the Pentium processor（为了更仔细地研究程序执行的方式，我们将为奔腾处理器编写一些简单的程序）
2) We will write this in a mnemonic form of the Pentium machine code.
3) This form is called assembly language.

2. Assembly language(汇编语言) and assembler(汇编器)



- The assembler program takes as input a program written in assembly language, and:
  – *Translate instructions into their binary **machine-code** form.*
  (把指令转换成二进制机器代码形式)
  – *Associates labels with memory **addresses**.* (将标签与内存地址相关联)
  – *Produces a binary **machine object code program**.*
  (生成一个二进制机器目标代码程序)
- *Associates **labels**(标签) with memory addresses.*

```
L1:     JZ      L2
        ...
        CALL    doD
        JMP     L1
L2:     ...
```

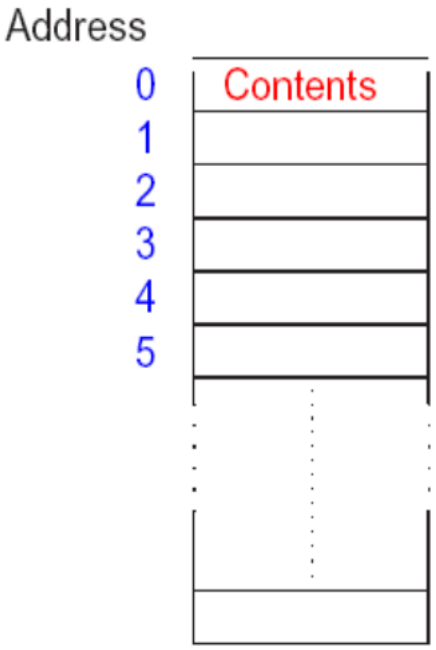
- Labels, for example, signify the beginning of a loop, the beginning of a block of memory cells(存储单元块), etc.
- A program has to be loaded into somewhere in main memory in order to be executed.
  – Programs must be relocatable.(程序必须是可以重定位的)
- *Binary machine object code program*
  – Also called object file(目标文件).
  – Contains binary machine code(包含二进制机器码).
  – May still need a linker to generate an executable or library by linking object files together. (仍需要 linker 来生成可执行文件或库)

3. Main memory, RAM

- A CPU may execute only instructions loaded in the main memory
  – RAM: **Random Access Memory**.
  – The name is to set it apart from **sequential access memory**(顺序存取存储器) (e.g. serial tape storage) with which you cannot access data stored in the last blocks without going through the first blocks of data.(在未通过第一个数据块的情况下，您无法访问存储在最后一个块中的数据)

4. RAM

1) The memory can be seen as a set of numbered storage elements, called **words**, each of which can contain some information.
   (memory 可以看作是一组被编号的存储元素，这些元素叫做 word，每一个 word 都可以包含一些信息。)
2) Each word is numbered with its address.(每个 word 都有它的地址编号。)
3) Any word of memory can be accessed "without touching" the preceding words (Random Access). 随机访问
4) Access time is the same for all the stored items.
   (所有存储项的访问时间相同)



5. Words, Bytes and Bits

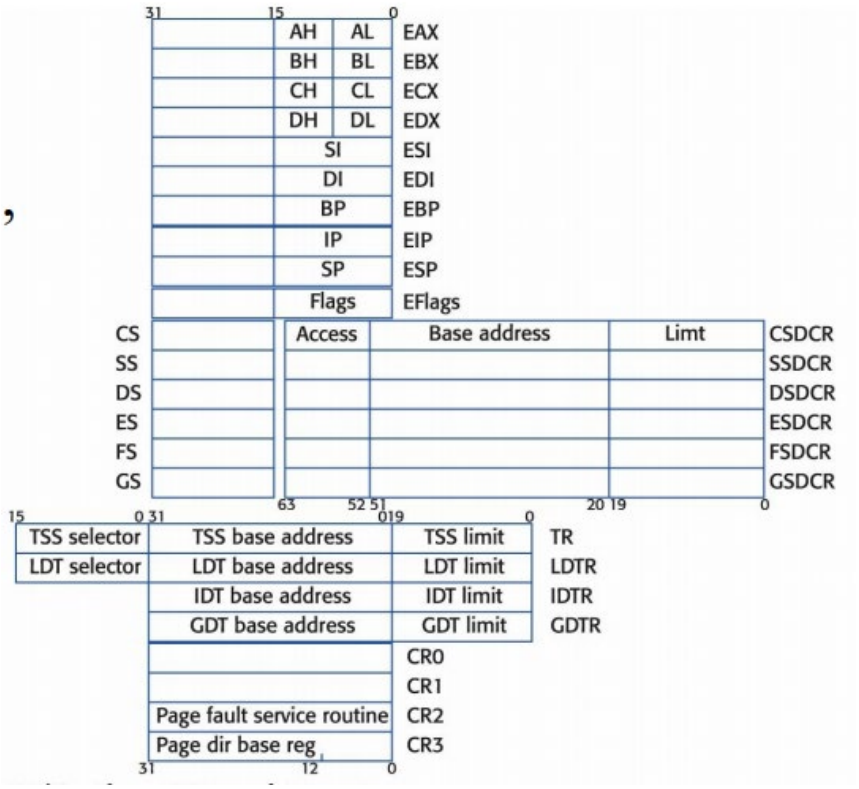
1) A word may be visualised as a set of more elementary storage elements (memory cells), called bits, arranged in a row.
   (一个 word 可以形象化为一组更基本的存储元素(记忆单元)，称为 bits，

排成一行)

2) The number of bits in the word may vary(变化) between different computers; 32 bits in word is common.
3) A standard unit of 8 bits called a byte.

6. CPU Registers in Pentium

- Not including the floating point registers, or the MMX (Multi-Media Extension) group. (不包括浮点寄存器或 MMX(多媒体扩展)组)



7. EAX register, or Accumulator (EAX 寄存器或累加器)

1) Used as a general-purpose data register(通用数据寄存器) during arithmetic and logical operations.
2) Some instructions are optimized(被优化) to work faster with the accumulator(累加器).
3) In some instructions (MUL and DIV) The accumulator is assumed(假想的). No need to write it explicitly(明确的).
4) It can be accessed in various ways as 8,16, or 32 bit chunks, referred to as AL, AH, AX, EAX. (它可以以各种方式被访问，如 8 位、16 位或 32 位块，称为 AL、AH、AX、EAX。)

8. Example of instruction using EAX

MOV EAX,1234H	Load constant value 4660 into 32 bit accumulator.
INC EAX	Add 1 to accumulator value.
MOV maxval,EAX	Store accumulator value to memory variable 'maxval'.
DIV CX	Divide accumulator by value in 16 bit register CX.

9. EBX: the Base register

- The Base register can hold addresses to point to the base of data structures, such as arrays in memory.(基址寄存器可以保存地址以指向数据结构的基址，例如内存中的数组)
- Example:

LEA EBX, marks	Initialize EBX with address of the variable 'marks'.
MOV AL,[EBX]	Get 1-byte value into AL using EBX as the memory pointer.

10. ECX: the Count register

- It is used as a counter in loops.
- Example:

MOV ECX,100	Initialise ECX as the FOR loop index.
……	
for1:	Symbolic address label.
……	
LOOP for1	Decrement ECX, test for zero, JMP back if non-zero.

11. EIP: the Instruction Pointer

- The Instruction Pointer (Program Counter) holds the address of the next instruction. (指令指针(程序计数器)保存着下一条指令的地址)
- Example:

JMP mylabel	Forces a new address into EIP.

12. ESI

- the Source Index register is used as a pointer for string or array operations. (源索引寄存器用作字符串或数组操作的指针)
- Example:

MOV AX,[EBX+ESI]	Get one 16-bit word using Base address and Index register.

13. EDI

- The Destination Index register is used as a pointer for string or array operations.(目标索引寄存器用作字符串或数组操作的指针)
- Example:

MOV EAX,[ESI] MOV [EDI],EAX	Moves one 32-bit word from source to destination.

14. EBP: The Stack Base Pointer.(栈基指针)

15. ESP: The Stack Pointer.(堆栈指针)

Lecture 9

1. CPU Registers in Pentium

- EAX---AH&AL
- EBX---BH&BL
- ECX---CH&CL
- EDX---DH&DL
- ESI---SI
- EDI---DI
- EBP---BP
- EIP---IP
- ESP---SP
- EFlags---Flags

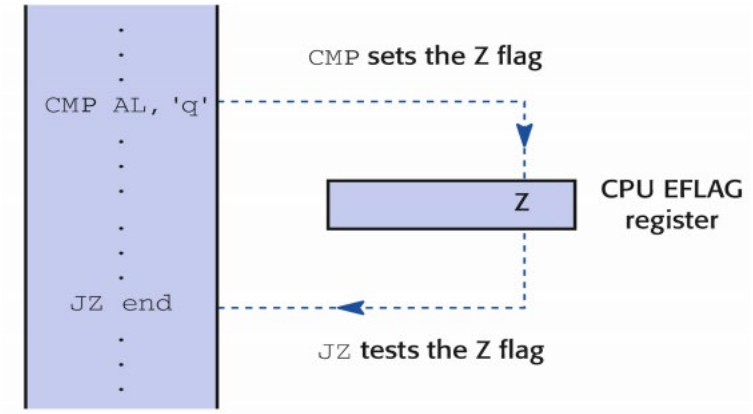
2. CPU status flags(CPU 状态标志)

1) **EFLAG**: The Flag register holds the **CPU status flags**.
2) The status flags are separate bits in EFLAG where information on important arising conditions such as **overflow** or **carry bits**, is recorded.
   (状态标志是 EFLAG 中单独的位, 其中记录了有关重要出现条件(如溢出或携带位)的信息。)
3) A way of communication between one instruction and the subsequent instructions.
   (一条指令和后续指令之间的通信方式)
4) Most often used flags are:
   - S: sign
   - Z: zero, result being zero
   - C: carry, indicates an arithmetic carry(进位，表示算术进位)
   - O: arithmetic overflow error(算术溢出错误)
     ---The addition of two large positive numbers which may give a negative result is a two's complement system.(两个大的正数相加可能会产生负数，这就是二进制补码系统)
     Example:

     - 0111011111111111
     - +0110111101110111
     - 1110011101110110

     - OF will be set.

3. How CPU flags are operated(CPU 标志的运行方式)



- CMP: compare two values , subtract with no result(不影响 result), only setting flags.
- JZ: conditional jump according to the Z flag.(依据 Z 标志 jump)

4. Inline Assembler(内联汇编)

1) Inline means you can insert assembly codes directly into C/C++ programs, compile and execute them.
2) Minimal knowledge of C/C++ is assumed. (无需过多的 C/C++语言基础)
3) Inline assembly code can refer to C or C++ variables by name(可以直接调用 C/C++ 中的变量名):

```
_asm mov eax,var      ; stores the value of
                      ; var in EAX.
```

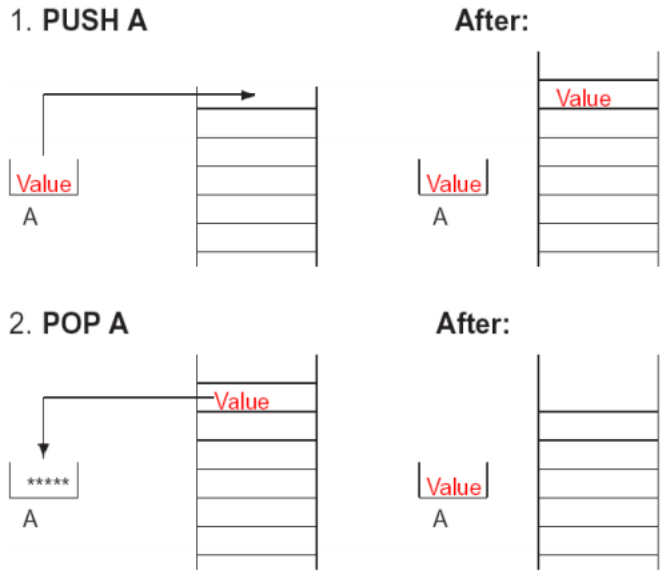
4) This example illustrates two more aspects:
   - – One can use **_asm** without brackets, in that case it works only for one line. (只有一行汇编语句时可不加大括号)
   - Semicolon ; (分号)is used for the comments in the assembly program. Alternatively, one can use // for the comments.
   - An **_asm { … …}** block can call C functions, including C library routines.
   - We use C library routines **scanf** and **printf** for input and output in our programs. （我们使用 scanf 和 printf 作为输入和输出的语句）
   - To pass arguments to **printf** we use a stack. （为了将参数传递给 printf，我们使用了一个堆栈）

5. printf & scanf 的例子

```
int x;
int day, year;
char monthname[20];
printf("%d\n", x);          /* how many parameters? */
scanf("%d %s %d", &day, monthname, &year);          /* input parameters */
```

6. Stack

1) Stack is a memory arrangement (data structure) for storing and retrieval information (values).（堆栈是一种存储和检索信息（值）的内存排列（数据结构））
2) The order of storing and retrieving(检索) the values for the stack can be described as LIFO (last in, first out).
3) The ESP register stores the address of the item that is on top of the stack.
   （ESP 存储器存放的 address 是 stack（堆栈顶部的 address））
4) Example of an alternative memory arrangement is a queue: FIFO
5) Every stack is equipped with two operations:
   - Push
   - Pop.



6) Stack is a simple but useful data structure.(Stack 是一种简单但有用的数据结构)
7) Almost any assembly language has special instructions for implementing a stack.
   (几乎任何汇编语言都有实现堆栈的特殊指令)
8) In inline assembly language there are **PUSH** and **POP** instructions.
   (在内联汇编中我们使用 POP 和 PUSH)
9) PUSH and POP operations use the stack pointer register ESP to hold the address of the item on top of the stack.
   (PUSH 和 POP 使用 ESP 所储存的 address(堆栈的顶端地址))

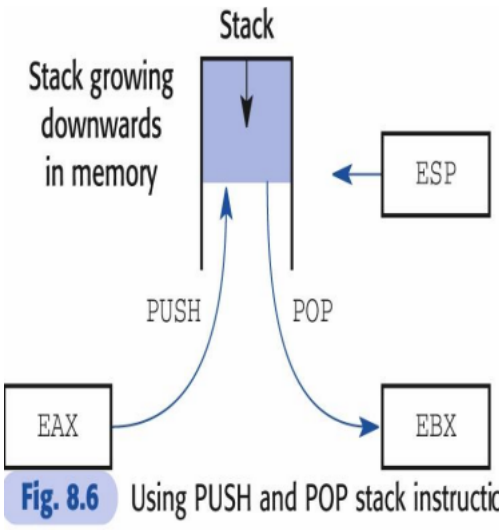
7. Assuming an upside-down stack… PUSH and POP

1) Assume an upside-down stack in memory.(假设一个上下颠倒的堆栈在内存中)

2) **PUSH** instruction works:
   - First by decrementing(消耗) the address in ESP. （首先减少 ESP 中地址（值））
   - Then using the ESP address to write the item into stack. （其次利用 ESP address 把自己写入堆栈中）

3) POP instruction works:
   - The item is read using ESP address.(利用 ESP 中地址读取堆栈中值)
   - The ESP address is incremented by a correct amount to remove the item from the stack.(ESP 地址增加正确的数量，从栈中移除该项)(换句话说就是把枪口对准下一位目标抛弃前任，寻找下一个目标)

8. Using PUSH and POP




```
PUSH EAX  ;push 32bit word in A into stack
CALL print;do something
POP EBX   ;pop the 32bit word from stack
```

Fig. 8.6 Using PUSH and POP stack instructions.

© Pearson Education 2001

9. Adjusting stack pointer(校准/调整堆栈指针---ESP)

- Sometimes the stack pointer has to be adjusted explicitly by the programmer.
  （有时必须由程序员显式地调整堆栈指针）

```
ADD ESP,4        ; take 4 bytes off the stack
                 ; (see example in the Hello World program).

SUB ESP,256      ; create 256 bytes of stack space.
```

10. Stack as the temporary store（堆栈作为临时存储）

```
/* demonstration of the use of asm instructions within a C program */
#include <stdio.h>
#include <stdlib.h>
int main (void)
{
   char format[] = "Hello World\n";     // declare variables in C
   _asm{
        mov ecx,10                      // initialize loop counter
    Lj: push ecx                        // loop count index saved on stack
        lea eax,format                  // load the address of the array
        push eax                        // address of string, stack parameter
        call printf                     // use library code subroutine
        add esp,4                       // clean 4 byte parameter off stack
        pop ecx                         // restore loop counter ready for test
        loop Lj                         // dec ECX, jmp back if ECX nonzero
   }                                    // back to C
   return 0;
}
```

1) In the above example stack was used as a 'scratch-pad' temporary store to keep the value of loop counter and free ECX register for other use.（In the above example stack was used as a 'scratch-pad' temporary store to keep the value of loop counter and free ECX register for other use.）
2) That is a common use of the stack as a temporary store for variables (values).
（这是堆栈作为变量(值)临时存储的常见用途）

11. Stack for passing parameters

```
#include <stdio.h>
#include <stdlib.h>
int main (void)
{
   char format[] = "Hello World\n"; // declare variables in C
   _asm{
        mov ecx,10              // initialize loop counter
    Lj: push ecx                // loop count index saved on stack
        lea eax,format          // load the address of the array
        push eax                // address of string, stack parameter
        call printf             // use library code subroutine
        add esp,4               // clean 4-byte parameter off stack
                                // same as printf ("Hello World\n") in C
        pop ecx                 // restore loop counter ready for test
        loop Lj                 // dec ECX, jmp back if ECX nonzero
   }                            // back to C
   return 0;
} What happens if we don't "add esp,4" after finishing the call to 'printf'?
```

1) **printf** routine needs:
- extra information on what to print and how to print.(被告知 print 什么和怎样 print)
- It should be passed as parameters.（被当作参数传递）

2) It is done via stack:（通过堆栈实现）
   - **push eax** places the address of the string to be printed on the stack.
   - **call printf** reads the address from the top of the stack and prints the string, but it does not remove this address from the stack.
   - **add esp,4** cleans the top of stack (remove address).

补充：不同的 flag

O	Arithmetic overflow error
D	Direction of accessing string arrays
IE	External interrupt enable
T	Trap, single-step debugging, generates an INT #1 after each instruction
S	Sign, MS bit value
Z	Zero, result being zero
A	Auxiliary carry, used by BCD arithmetic on 4 LS bits
P	Parity, operand status
C	Carry, indicates an arithmetic carry or borrow result

Fig. 7.9 CPU status flags.

Lecture10

1. Structure of instructions

1) The binary codes of almost all instruction contain three pieces of information: (几乎所有指令的二进制码都包含三条信息)
   - The **action** or operation of the instruction. 指令的动作或运算
   - The **operands** involved (where to find the information to operate with). 操作对象
   - Where the **result** is to go. **结果**要被分配到哪里
2) Machine instructions are encoded with **distinct bit fields in the prefix** to contain information about(机器指令在前缀中用不同的位域编码，以包含有关的信息)
   - The operation required.(需要的操作)
   - The location of operands and results. (操作数和结果的位置)
   - The data type of the operand.(操作数的数据类型)
3) Pentium instructions can be from 1 to 15 bytes long, depending on:
   - The operation required
   - The addressing modes employed(采用的寻址模式)

2. Addressing modes(寻址模式)

1) Addressing mode—**benefit 好处**
   - The way of forming operand addresses.(构成操作数地址的方法)
   - More technically, the way one can compute an effective address. (计算有效地址)
     --- Effective address is the address of operand used by instruction.
     有效地址：指令所使用的操作地址
   - Offering various addressing modes support better the needs of HLLs when they need to manipulate large data structures.
     (提供各种寻址模式，可以在需要处理大型数据结构时更好地满足 HLL 的需求)
2) **Immediate (operand) mode**.（立即（操作）模式）
   Example: *mov eax,104* **值到寄存器**
3) **Data Register Direct**:（This is the fastest to execute.）这个是速度最快的运行办法
   Example: *mov eax, ebx* **寄存器到寄存器**
4) **Memory Direct**
   Example: *mov eax, a* **内存到寄存器**
   • Here 'a' is a variable, stored in memory and the instruction contains the address of this variable. （这里 a 是一个变量，存储在内存中，指令中包含了这个变量的地址）
   • Decode the address into a temporary register within the CPU, and then go to the memory location to read the data into the CPU.（将地址解码到 CPU 内的临时寄存器中，然后转到内存位置以将数据读入 CPU）
5) **Address Register Direct**:
   Example: lea eax, message1

• The instruction, contains the address of 'message1' variable, which is loaded into eax register after the execution of the instruction.（指令包含" message1"变量的地址，该地址在执行指令后加载到 eax 寄存器中）

• Commonly used to initialise pointers which can then reference data arrays.（通常用于初始化指针，然后可以引用数据数组。）

6) Register Indirect

Example: mov eax, [ebx]

该指令将存储在 ebx 中的地址的内存位置的**内容**复制到 eax 寄存器

```
/* Handling arrays with  Register Indirect addressing mode */
    ...
    int myarray[5];        // declaration of an array of integers
    myarray[0] = 1;        //
    myarray[1] = 3;        //
    myarray[2] = 5;        //  initialise the array
    myarray[3] = 7;        //
    myarray[4] = 9;        //
    _asm
    {
      ...
      lea ebx,myarray      // address of the array (its 0th element) is saved in ebx
      mov ecx,5            // size of the array is saved in the counter
      mov eax,0            // eax will be used to store the sum, initialise to 0
loop1: add eax,[ebx]       // read an element of the array at the address stored in ebx
      add ebx,4            // int occupies 4 bytes (32 bits),
                           // so to read next element increase the address in ebx by 4
      loop loop1           // end of the loop
     }
```

7) Indexed Register Indirect with displacement:变址操作

Example:

```
mov eax,[table+esi]
mov eax,table[esi]
```

• The symbol 'table' is the address of the array base.

• effective address= address 'table + esi(address)

```
/* Handling arrays with  Register Indirect with Displacement addressing mode */
    ...
    int myarray[5];        // declaration of an array of integers
    myarray[0] = 1;        //
    myarray[1] = 3;        //
    myarray[2] = 5;        // initialise the array
    myarray[3] = 7;        //
    myarray[4] = 9;        //
    _asm{
       mov ecx,5           // size of the array is saved in the counter
       mov eax,0           // eax will be used to store the sum, initialise to 0
       mov ebx,0
loop1:add eax,myarray[ebx] // read an element of the array at the
                           // address myarray + ebx
      add ebx,4            // int occupies 4 bytes (32 bits),  so to read
                           // next element increase the address in ebx
                           // by 4
      loop loop1           // end of the loop
     }
```

Lecture 11

1. Output in inline assembly

```
…
char format[] = "Hello World\n"; // declare variables in C
…
…
lea eax,format      // load address of the string 'format' into eax
push eax            // address of string, stack parameter
call printf         // use library code subroutine
add esp,4           // clean 4 byte parameter off stack
```

和下面这憨批玩意是一样的

printf("Hello World\n"); //C 语言中的 printf 语句

2. Printing numbers in assembly code

printf("%d",myint); C 语言中的 printf

汇编的语法:

1) Push the second parameter (integer variable to the stack).
2) Push the first parameter (actually, address of the string) to the stack.
3) Call printf routine
4) Clean up top two positions in the stack.

```
/* demonstration of the use of asm instructions within C prog  */
#include <stdio.h>
#include <stdlib.h>
int main (void)
{
   char format[] = "%d\n";  // declaration of the format string to
                            // be used in printf function as the first parameter
   int myint = 157;  // declaration of an integer variable
   _asm
   {
     push myint          // push the value of the variable onto the stack
     lea eax,format      // address of the format string is saved in eax
     push eax            // push the address of the string to the stack
     call printf  // call printf, it will take two parameters from the stack
     add esp,8           //clean up top two positions in the stack
   }
   return 0;
}
```

3. Input in inline assembly

C 语言的 scanf 语句：scanf("%d",&input);
汇编的骚操作：

```
...
char format[] = "%d";
int input;     //declaration of an integer variable for user's input
_asm
{
  lea eax,input
  push eax
  lea eax, format  //address of the format string is saved in eax
  push eax         // push the address of the string to the stack
  call scanf       // call scanf, it will take two parameters from
                   // the stack scanf(%d,&input);
                   // user's input will be put in the 'input' variable
  add esp,8        // clean top two positions in the stack
}
...
```

4. More about printf

– %c print a character
– %d, or %i print a signed decimal number
– %s print a string of characters

5. More about scanf

– %c read a single character
– %d read a signed decimal integer
– %s read a string of characters until a white space or terminator (blank, new line, tab) is found

6. Controlling program flow

1) Very few programs execute all instructions sequentially, from the first till the last one.
2) Usually, one needs to control the flow of the program:
   - Jump from one point to another, often depending on some conditions.
   - Repeat some actions while some condition is maintained , or until some condition is reached.
   - Passing control to and from procedures

7. Unconditional jumps（无条件 jump）

1) JMP instruction transfers control **unconditionally** to another instruction.
2) It has a syntax(句法规则):
   JMP<address of the target instruction(目标指令的地址)>
3) The address of the target instruction is given by its label.

```
label1:    …
           …
           …
           jmp label1

label2:    …
           …
           …
           jmp label2
           …
keep-going:
```

- ‘**label1**’, ‘**label2**’ and ‘**keep-going**’ are all labels.
- Unconditional jumps skip over code that should not be executed.

8. Conditional jumps

1) Conditional jumps work as follows:
   - First test the condition.
   - Then jump if the condition is true or continue if it is false.
2) There are more than 30 jump instructions:
   - Two of them, JCXZ and JECXZ, test whether the counter register CX, or ECX is zero.
   - Remaining jump instructions test the status flags.
3) Example:

```
       …
       …
       …
       jecxz finish
       mov eax,inp
       …
       …
       …
finish: add esp,4
```

- When ‘**jecxz finish**’ instruction is executed: if **ecx** register contains 0, then the next instruction to execute is
  add esp,4
- Otherwise,
  mov eax,inp
  is executed.

9. Jumping based on status flags

Instruction	Jump if
JC/JB	Carry flag is set (=1)
JNC/JNB	Carry flag is clear (=0)
JE/JZ	Zero flag is set (=1)
JNE/JNZ	Zero flag is clear (=0)
JS	Sign flag is set (=1)
JNS	Sign flag is clear (=0)
JO	Overflow flag is set (=1)
JNO	Overflow flag is clear (=0)

10. Jumps Based on Comparison of Two Values

1) The CMP instruction is the most common way to test for conditional jumps.
2) It compares two values without changing them, while it changes the status flags according to the results of the comparison.(它比较两个值而不更改它们，同时根据比较结果更改状态标志)
3) For example, if the value of eax and ebx are the same then the execution

cmp eax,ebx

will set zero flag Z=1.

11. Jumps Based on Comparison

Instruction	Jumps if (assuming execution just after CMP)
JE	The first operand (in CMP) is equal to the second operand.
JNE	The first and second operands are not equal.
JG	First operand is greater.
JLE	First operand is less or equal.
JL	First operand is less.
JGE	First operand is greater, or equal.

Example:

```
add ax,input ;      Add input to AX
cmp ax,0 ;          Compare AX with 0
jge label1 ;        Equivalent to: If ( AX >= 0 ) go to label1
jl label2 ;         Equivalent to: If ( AX < 0 ) go to label2
```

Lecture 12

1. Controlling program flow: Loops

1) Simple loop:
   - Loop<label>
     Automatically decrements ECX
       when ECX = 0, the loop ends
       when ECX is not 0 it jumps to<label>

2) Example:

```
mov ecx, 200          ; set counter
next:…                ; do the task here
… loop next           ; jump to the label 'next'
…                     ; continue after looping
```

3) Another example – loop upon two conditions:
   ---While EAX is not equal to EBX AND not 200 times yet:

```
mov ecx,200       ; Set counter
next: …           ; Set label
…                 ; Do something
cmp eax,ebx       ; Are EAX and EBX the
                  ; same? Or 200 times already?
loopne next       ; No? Go to next.
…                 ; Yes? Continue
```

2. Implementing higher-order constructs: conditional statements(实现高阶结构:条件语句)

- If statement

- In Java:

```
if (c > 0)
  pos = pos + c;
else
  neg = neg + c;
```

- Equivalent in the assembly code:

```
      mov eax,c
      cmp eax,0
      jg positive
negative:add neg,eax
      jmp endif
positive:add pos,eax
endif:…
```

- Equivalent in the assembly code:

```
      mov eax,c
      cmp eax,0
      jle negative
positive:add pos,eax
      jmp endif
negative:add neg,eax
endif:…
```

- While statement

- In Java:

```
while (fib2 < 1000)
{
   fib0 = fib1;
   fib1 = fib2;
   fib2 = fib1 + fib0;
}
```

- Equivalent in the assembly code:

```
while:mov eax,fib2
      cmp eax,1000
      jge end_while
      mov eax,fib1
      mov fib0,eax
      mov eax,fib2
      mov fib1,eax
      add eax, fib0
      mov fib2,eax
      jmp while
end_while: …
```

- Switch-Case statement

- In Java:

```
switch (num){
  case 1:  … ;
     break;
  case 2:  … ;
     break;
}
```

- Equivalent in the assembly code:

```
mov eax,num
     cmp eax,1
     je case_1
     cmp eax,2
     je case_2
     jmp end_case
case_1:…
     jmp end_case
case_2:…
end_case:…
```

Lecture 13

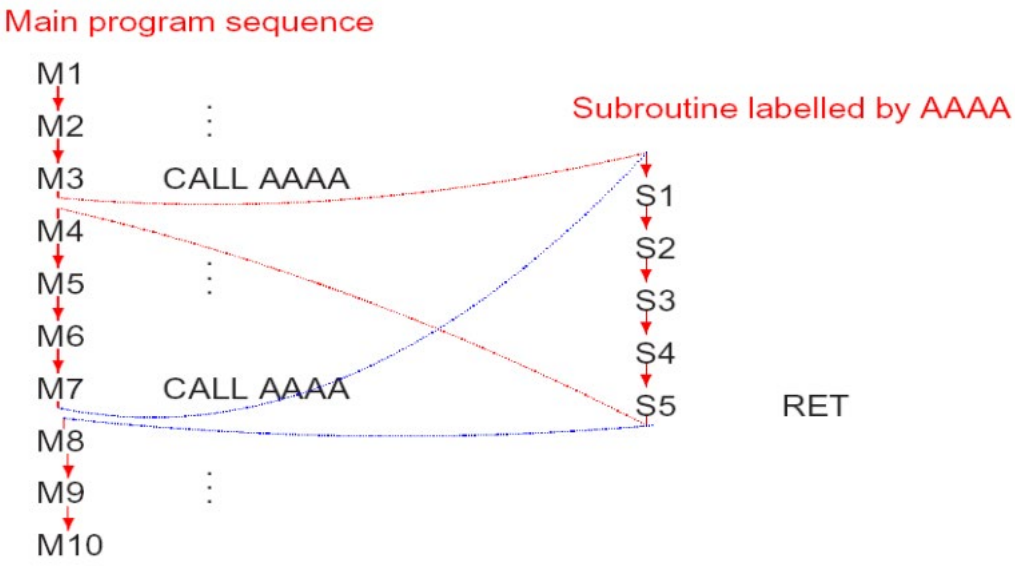
1. Subroutines(子程序)

1) A **subroutine** is a general term. In different programming languages it may be called differently:
   - **Procedure** in Pascal.
   - **Function** in C.
2) a subroutine is a part of the code, which can be used repeatedly within the program that is being executed.(子程序是代码的一部分，可以在正在执行的程序中重复使用)

3) an example: “scanf”&&”printf”

4) 优点:
   - Save the effort in programming
   - Reduce the size of programs.
   - Share the code
   - Encapsulate(封装), or package, a particular activity.
     --- Hide complications(困难) from the user.
   - Provide easy access to tried and tested code.
   - Facilitate code reuse.

5)



6) Subroutines in assembly language

- Declaration(声明) of a simple procedure looks as follows:

```
label PROC
        ...
        ...
        ...
        RET             ; return
label ENDP
```

- The procedure can be called by the instruction:
  call label
- Return from the subroutine
  ------ RET 使得子程序结束运行，重新回到主程序中
- CALL 的工作原理
  ---Records the current value of EIP (Instruction Pointer) as the return address.
  (将 EIP(指令指针)的当前值记录为返回地址)
  --- Places the required subroutine address into EIP (instruction Pointer) , so the next instruction to execute is the first instruction of the subroutine.
  (将所需的子例程地址放入 EIP(指令指针)中，因此要执行的下一条指令是子例程的第一条指令)

2. Nested calls（call 嵌套）

```
    ...
     :
  CALL SUB1       * call first subroutine
     :
SUB1: ...
     :
  CALL SUB2       *call second subroutine
     :
  RET
SUB2: ...
     :
  RET
```

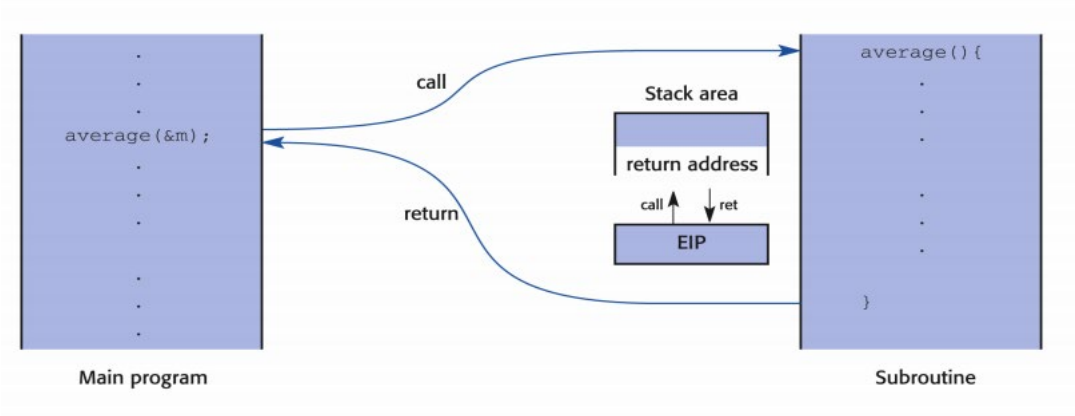
1) In the upper example we need both return addresses stored at the same time.
2) The more deep nesting of subroutine calls, the more space do we need to store all return addresses.(越深层的嵌套需要的用来储存 return address 的空间就越大)
3) An elegant solution is to use a stack to store all return addresses.（一个优雅的解决方案是使用堆栈来存储所有的返回地址）

3. CALL and RET working together

1. CALL does the following:
   - Records the current value of EIP (Instruction Pointer) as the return address: copy the address from EIP to the stack (PUSH). (将当前的 EIP 值作为 return address，并将其 push 到堆栈里去)

   - Puts the required subroutine address into EIP (Instruction Pointer) , so the next instruction to execute is the first instruction of the subroutine.（将要执行的子程序的地址塞入 EIP 中，以便于执行）

2. RET does the following:
   - Pop the last address stored in the stack and put it into EIP.(将上一个被塞入堆栈的地址取出，并放入 EIP 中)



Lecture 14

1. Parameters(参数)

1) The simplest kind of subroutine performs an identical function each time it is called. Such a subroutine requires no further information to run.
（最简单的子程序在执行时每次调用都是执行一样的 function，也就是说不需要额外的信息）
2) But most subroutines require additional information, given in parameters.
（但是大部分的子程序还是需要外加的信息的（通过 parameter 来给））
--- For example, the printf subroutine requires information what to print and how to print.
3) Stack can be used to pass parameters to subroutines (lecture 11), and to store return addresses of subroutine calls (lecture 13).
4) In general, parameters can take a number of forms, depending on the nature of the information required by the subroutine.（一般来说参数可以采取很多种形式，这取决于子程序的需求）
5) Two main forms of parameters:
   - **Value parameters.**
   - **Reference parameters**

2. Value parameters

1) In many cases, the additional information required by a subroutine will be a simple value (or values) of some type(s):（在许多情况下子程序所需要的额外信息是一个简单的值）
--- For example, a numeric value, or ASCII code value, etc.
2) 输入子程序的值就叫做 parameters

3. Reference parameters（引用参数）

就是参数变成了对应的内存的地址

```
      …
      lea eax, first        ;
      lea ebx, second       ;
      call swap             ; calling
finish: …
  ;
swap proc                   ; subroutine which uses
      mov temp, [eax]       ; reference parameters
      mov [eax],[ebx]       ; passed via registers
      mov [ebx],temp        ;
      ret                   ;
swap endp
```

4. Passing parameters via registers（通过寄存器传递参数）

1) The simplest method of passing parameters into a subroutine is to use a register:
2) Both value and reference parameters can be passed using registers.两种参数都能被寄存器传输
3) But, this method would be too constraining!

5. Stack instead of registers

1) For subroutines with a **few** parameters one may pass these parameters (in address, or value form) using general-purpose registers.
   （对于带有少数参数的子程序，可以使用通用寄存器传递这些参数(以地址或值的形式)。）
2) However, in general, the number of registers available is very limited, and is not enough to implement subroutines with an arbitrarily large number of parameters.（当所需参数过大时，寄存器就显得很局限了）
3) Thus, we need to use the memory in order to implement parameter handling for subroutines.
4) The solution in most computers is **to use the stack for passing parameters and keeping the local variables.（大部分计算机的办法是使用堆栈来传递参数并且保存局部变量）**

6. Local variables（局部变量）

只能在子程序中被调用的就是局部变量（就像 JAVA 中被定义子啊 method 内部的变量）

7. Stack frame（栈框架）

1) The area of the stack which holds all data related to one subroutine call is called a stack frame.（和保存所有堆栈数据有关的子程序叫做 stack frame（栈框架））
2) This data includes:
   - Parameters of the subroutine.（子程序的参数）
   - Return addresses.（返回的地址）
   - Local variables.（局部变量）

8. EBP and ESP for stack frames

1) Because of the nested calls several stack frames may be present at the same time.（由于嵌套调用，可能同时存在多个堆栈帧。）
2) Two registers are used to work with stack frames:
   - EBP: The stack frame base pointer.（堆栈帧基指针）
     --- To indicate the base of the current stack frame（指示当前堆栈帧的基）
   - ESP: The stack pointer.（堆栈指针）
     --- To hold the address of the top of the stack（存储顶端值）

9. How does all this work?

1) ESP is always pointing to the top of the stack（ESP 总是指向堆栈的顶部）
2) EBP initially contains an address of the base of the stack（EBP 最初包含一个栈基地址）
3) **Just before and during the call of a subroutine the following happens:**
   - **The parameters are pushed on the stack.（参数被 push 进入堆栈）**
   - **The return address is pushed on the stack.（返回地址被 push 进堆栈）**
   - **The address stored in EBP is pushed on the stack.（EBP 中的地址也被 push 进堆栈）**
   - **A new stack frame is created.（新的栈构架被建立）**
   - **The current address of the top of the new stack frame is saved in EBP.（新栈帧顶部的当前地址保存在 EBP 中）**
   - **The local variables are installed on the new stack.（本地变量被安装在新的堆栈上）**
4) **Once the subroutine done its job：**
   - **Pop all local variables out of the stack.（将所有局部变量 pop 出堆栈）**
   - **Pop the previous EBP address from the top of the stack and restore it in EBP (stack frame base pointer).（取出之前的 EBP 值，并将其重新储存在 EBP 中）**
   - **Clean up parameters in the stack.（清除堆栈中的参数）**
   - **Pop the return address and save it in EIP.（pop 返回地址并且保存至 EIP 中）**
   - **Note: popping order is crucial.（pop 的顺序非常重要）**

The data(in Stack frame) includes:
– Parameters of the subroutine.
– Return address.
– Old stack pointer contents (EBP)
– Local variables.

Lecture 15

1. Recursive subroutines(递归子程序)

1) A recursive subroutine, or procedure, is one that may in some circumstances calls itself to perform some subsidiary task.(递归子程序或过程在某些情况下可能调用自己执行某些辅助任务)
2) For example a subroutine SUBR may CALL SUBR.
3) Recursion may appear in a more subtle form of mutual recursion.
   （递归可能以更微妙的相互递归形式出现）
   - SUBR1 calls SUBR2, and
   - SUBR2 in turn calls SUBR1.

2. Examples of recursive definitions (procedures)

1) Factorial:
   -factorial(1) = 1.
   – factorial(n) = n * factorial(n-1).

```
  factorial PROC   //input n in eax
      push eax     // push current value onto the stack
      dec eax      // decrease the value of n
      jz finish    // if it is zero go to finish
      call factorial  // otherwise call factorial
      push eax        // push the result of last
                      // factorial's call to the stack
      call multiply   // call multiply subroutine
      ret             // return
finish: pop eax       // pop the parameter from the
                      // stack into eax
      ret             // return with the result in eax
  factorial ENDP      // side effect?
```

```
multiply PROC         // input from two top values on the stack
       pop eax        // pop a value from the stack to eax
       mov aux, eax   // move this value to the auxiliary variable
       pop eax        // pop one more value from the stack
       mul eax, aux   // multiply these two values
       ret            // return the result in edx:eax
multiply ENDP
```

Lecture 16

1. Data codes – numeric and character

• Within the computer the binary number system is a system of choice, both for data storage and for internal processing of operations.(在计算机中，二进制数字系统是一种可供选择的系统，既用于数据存储，也用于内部操作处理)

2. Alphanumeric data(字母数字数据)

1) The majority of the data originally comes in the form of letters in alphanumeric data(大部分数据最初以字母数字数据的形式出现)
   - Alphabet 字母
   - Numbers 数字
   - punctuation 标点符号

3. Numbers vs. text

1) Numbers (consisting of numeric characters) are often processed differently from text.（数字(由数字字符组成)的处理方式通常与文本不同）
2) But still they must be entered into computer just like any other alphanumeric characters, one digit at a time.（但是，它们仍必须像任何其他字母数字字符一样输入计算机，一次输入一个数字）
3) Conversion between alphanumeric representation of numbers and special representation of numbers is done usually in a program (software).（字母数字表示和特殊数字表示之间的转换通常在程序(软件)中完成）
   - The method *valueOf(double)* in *java.lang.String* class converts a double float number to its string representation.
   - The method *java.lang.Integer.parseInt(String)* converts a string into the integer value that it represents.

4. Number representations（数字表示）

1) We have seen that a computer stores all data in binary form.（计算机存储所有的数据用二进制）
2) To store numbers we need an **encoding scheme**, which would allow us to encode:（为了存储数字，我们需要一个编码方案，它允许我们进行编码）
   - The **algebraic sign** of numbers (+/-)（数字的代数符号 （+/-）。）
   - **Decimal point（小数点）** that might be associated with a fractional number.（可能与小数关联的小数点。）
3) One solution
   - Store the number as a string of characters
   - However, it is not practical for calculations.
4) Unsigned integers can be represented directly in binary form（无符号整数可以直接用二进制形式表示）

5. Two categories of integer data

1) Unsigned integers

- **Unsigned binary representation** – just store any whole number in its binary representation（直接存储就好了）
- Thus, a byte can store any unsigned integer of value between 0 and 255.（因此一个 byte 可以存储 0~155 之间的值）
- A 32-bit word can store **unsigned integers** in a range 0 – (2^{32} -1) = 4,294,967,295.（一个 32bit 的 word 可以存储 4,294,967,295）
- Multiple storage locations (words) can be used to represent bigger unsigned integers.（可以使用多个存储位置(字)来表示更大的无符号整数）

- **Binary-coded decimal (BCD):**

  Digit-by-digit binary representation of original decimal integer.（原始十进制整数的逐位二进制表示）

  - ✓ Each decimal digit（十进制数字） is individually（分别的） converted to binary.
    --- This requires **4 bits per digit** (not all 4-bits patterns are used).
  - ✓ Example:
    ---Decimal value 68 is presented in BCD as 01101000.
  - ✓ One byte can store unsigned integers in a range 0-99 under BCD encoding

- Binary vs. BCD representation
  - ✓ BCD is less economical than binary representation(CD 不如二进制表示经济)
  - ✓ Calculations in BCD are more difficult.
  - ✓ But, translation between BCD and character form is easier.
  - ✓ Binary representation is more common, BCD is used for some business applications.

2) Signed integers

- Sign-and-magnitude representation.(符号和大小表示)
- Complementary representations（补码表示）

6. Sign-and-magnitude representation

1) It is representation of signed integers by **a plus or minus sign and a value**.
2) Agreement.（协议）
   - **Left-most** bit represent a sign, e.g., 0 stands for + and 1 stands for -.
3) Example:
   - 00100111 represents 39
   - 10100111 represents -39
   - 00000000 represents 0
   - 10000000 represents 0
4) Calculations are more difficult than in the case of binary (unsigned) representation

5) There are two different binary codes for the number 0: 00000000 and 10000000 (8-bit codes).
6) Positive range of the signed integer is one-half as large as the unsigned integer of the same number of bits.（带符号整数的正值范围是相同位数无符号整数的一半大）（对负数一样适用）
7) Thus, 8-bits can represent the numbers from -127 to 127 (0 being represented twice)

7. Complementary representations（补码表示）

1) For most purposes, computers use different methods of representing signed integers known as complementary representations (conventions).（在大多数情况下，计算机使用不同的方法来表示有符号整数，这些方法被称为互补表示(协定)）
2) Two's binary complementary representation is the most common（最常见的是二进制的二进制补码表示法）
   Thus, the numbers (3-digit sequences):
   --- From 000 to 499 are representing themselves.
   --- Any number x from 500 to 999 represents negative number **-(1000-x),** that is, negated complement to 1000.
   --- Number 0 is not represented twice.
   对于 n-digit 的处理公式为 y=-(n*9+1-x)
3) Example(x-y)

- Assume $x \geq y$.
  - Instead of subtracting y from x, the complement of y is added to x and the leading '1' of the result is discarded.
- Example (working with 3 digits).

373	(x)	→	373	(x)	
- 218	(y)		+ 782	(10's complement of $-y$)	
?			1155		

  - The first '1' digit is then dropped, giving 155, the correct answer.

- In the case $x < y$.
  - There will not be a '1' digit to cross out after the addition.
- Example (working with 3 digits).

185	(x)	→	185	(x)	
- 329	(y)		+ 671	(10's complement of $-y$)	
?			856	(10's complement of the result -144)	

  - Representing negative numbers in 10's complements.

Lecture 17

1. Examples of addition in 3 digits

10's complementary codes	Actual values
699	- 301
400	400
-------	------
1099	99
099	

简单来说就是先转换基础数据，在进行运算。

2. Subtraction

和加法类似，转换就完事了。

3. Overflow testing（溢出测试）

- Simple rule: If both inputs to an addition have the same sign, and the output sign is different, overflow has occurred.（如果加法的两个输入都具有相同的符号，而输出符号不同，则发生溢出）

4. Positive and negative

1) In 10's complement system the range is unevenly divided between positive and negative integers.（在 10 的补码系统中，范围在正整数和负整数之间平均分配）
2) For example, 4-digit complement system represents all the integer values between -5000 and 4999.

5. Two's complement（二进制补码）

1) Numbers that begin with 0 are considered to be **positive** representing themselves.（以 0 开头的数字被认为是正的，代表它们自己）
2) Numbers that begin with 1 are representing **negative** numbers.（以 1 开始的数字表示负数。）
3) A complement of the number in 2's complementary n-digit encoding can be found in one of two ways.（在二进制下的补码可以通过两种方式来寻找）
   - Either subtract（减去） the value from the modulus（从模中减去值）
   - Invert all bits and add 1 to the result.(反转所有的 bits 并且加 1)

6. Numerical types in Java

- **byte** 8-bit: integers from -2 8-1 to 28-1 -1
- **short** 16-bit: integers from -2 16-1 to 216-1 -1.
- **int** 32-bit: integers from -2 32-1 to 232-1 -1.
- **long** 64-bit: integers from -2 64-1 to 264-1 -1.

Lecture 18

1. Floating Point Numbers（浮点数）

1) It is not always possible to express numbers in integer form.（数字的表示不总是整数形式）
2) Real, or floating point numbers are used in a computer when：
   - The number to be expressed is outside of the integer range of the computer, like 5.375×10^{25}.（要表示的数字超出计算机的整数范围）
   - Or, when the number contains a fraction, like 345.0256（或者包含了小数点）

2. Exponential notation (base 10)（指数表示法（基数 10)）

1) In general, this notation represents numbers in a form $a \times 10^b$.

3. Components of exponential notation（指数符号的组成）

1) The sign of the number.（数字的符号）
2) The magnitude of the number, known as the mantissa.
   （数字的大小，称为尾数）
3) The sign of the exponent（指数的符号）
4) The magnitude of the exponent.（指数的大小）
5) Two additional pieces of information:
   - The base of the exponent (e.g., 10 or 2).（指数的基基底）
   - The location of the decimal point.（小数点的位置）

4. Floating point formats（浮点格式）

1) Any format for floating point numbers should specify how the components of an exponential notation are stored (in a word, or several words).（浮点数的任何格式都应指定指数符号的分量存储方式（在单词中或几个单词中））
2) The **base of the exponent** and the **location of the binary point** are standardised（标准化） as part of the format and, therefore, do not have to be stored at all.（指数的基地和二进制小数点的位置是标准化作为格式的一部分,因此,不需要存储）
3) Example：Suppose, that the standard code consists of space for seven digits and a sign：

   SEEMMMMM

   ---So, we have two digits for the **exponent** and 5 digits for the **mantissa（**小数部分**）**.
4) Trade-off: precision (mantissa) vs. range (exponent).
   权衡：精度(尾数)vs 范围(指数)
5) Most commonly, the mantissa is stored using **sign-magnitude** format.（通常，尾数使用符号格式存储）

5. Excess-n notation for the exponent

1) Excess-50 notation for the 2-digit decimal（十进制） representation of the exponent:

Representation	$0 \ldots \quad 49 \mid 50 \ldots 99$
Exponent being represented	$-50 \ldots -1 \mid 0 \ldots \quad 49$

2) Offset the value of the exponent by a chosen amount (here it is 50).（用选定的值(这里是 50)抵消指数的值）
3) It is simpler to use for exponents than the complementary form.（相比于 complementary 形式 exponents 更简单）
4) Thus, 5-digit excess-50 notation allows us a magnitude range of

$$0.00001 \times 10^{-50} < \ldots < 0.99999 \times 10^{+49}$$

--- We assume that the decimal point is located at the beginning of five-digit mantissa.（我们假定小数点位于五位数的尾数的开头）

6. Normalisation of floating point numbers（浮点数的规范化）

1) To maximise the precision for a given number of digits, numbers are, usually, stored **with no leading zeros**.（为了使给定数字的精度最大化，通常在存储数字时不带前导零）
2) Process of transformation the numbers into such a form is called **normalisation**.（将数字转换成这样一种形式的过程称为规范化）
3) Example:
   – A number: 0.00123×107
   – It's normalised form: 1.23×104

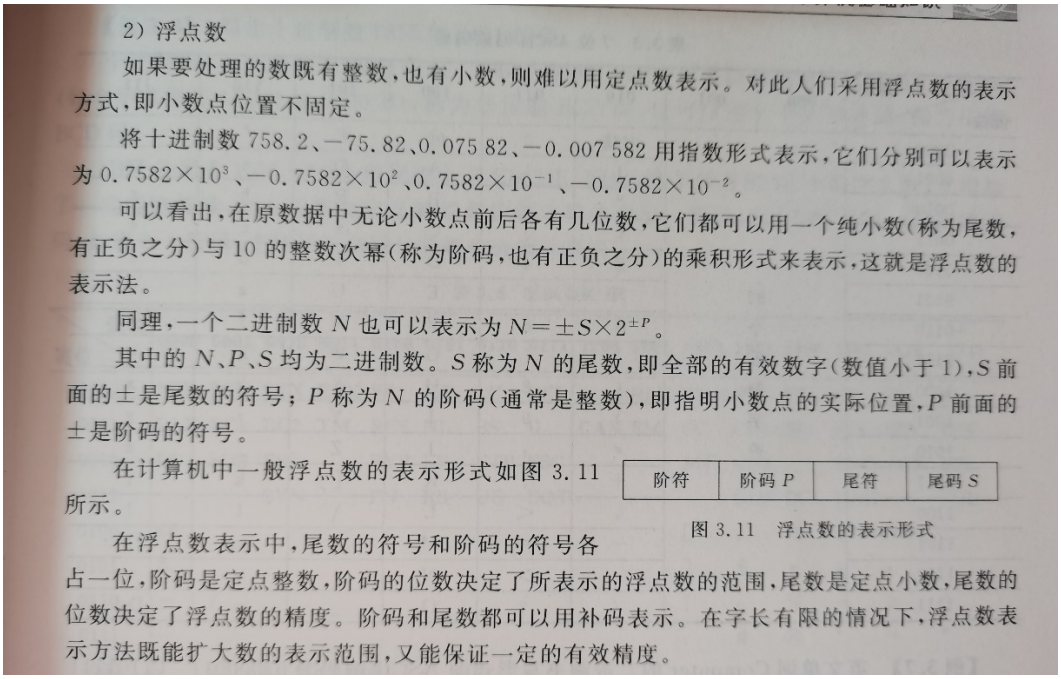
7. Floating point in the computer: binary representation

1) Typically, 4, 8 or 16 bytes are used to represent a floating point number.
2) Typical floating point format: 32 bits are used to provide a range of approximately 10-38 to 10+38 .
   - 1 bit is used for sign of mantissa.（1bit 用作尾数的符号）
   - 8 bits are used to store the exponent in excess-128 notation.（8bit 被用来储存 excess-128 格式的指数）
   - 23 bits are used for mantissa.（23bits 被用作储存尾数）

8. Floating point in binary

1) Assuming **normalised** representation one can omit the storage of most significant bit (it is always 1 !).
   （假设采用标准化表示，就可以忽略最高位的存储）
   ---So, 23 bits provide 24 bits of precision.（23 位提供 24 位的精度）
2) Binary point should be specified.（应该指定二进制点）
   - Most common choice is **after the most significant bit, i.e. 1.???**.
   - Notice, this bit itself is not stored!

9. 对于二进制浮点数转化为十进制



2）浮点数

如果要处理的数既有整数，也有小数，则难以用定点数表示。对此人们采用浮点数的表示方式，即小数点位置不固定。

将十进制数 758.2、−75.82、0.075 82、−0.007 582 用指数形式表示，它们分别可以表示为 0.7582×10^{3}、-0.7582×10^{2}、0.7582×10^{-1}、-0.7582×10^{-2}。

可以看出，在原数据中无论小数点前后各有几位数，它们都可以用一个纯小数（称为尾数，有正负之分）与 10 的整数次幂（称为阶码，也有正负之分）的乘积形式来表示，这就是浮点数的表示法。

同理，一个二进制数 N 也可以表示为 $N=\pm S\times2^{\pm P}$。

其中的 N、P、S 均为二进制数。S 称为 N 的尾数，即全部的有效数字（数值小于 1），S 前面的±是尾数的符号；P 称为 N 的阶码（通常是整数），即指明小数点的实际位置，P 前面的±是阶码的符号。

在计算机中一般浮点数的表示形式如图 3.11 所示。

阶符	阶码 P	尾符	尾码 S

图 3.11　浮点数的表示形式

在浮点数表示中，尾数的符号和阶码的符号各占一位，阶码是定点整数，阶码的位数决定了所表示的浮点数的范围，尾数是定点小数，尾数的位数决定了浮点数的精度。阶码和尾数都可以用补码表示。在字长有限的情况下，浮点数表示方法既能扩大数的表示范围，又能保证一定的有效精度。

(以下两个例子都是在 excess-128 下的)

- Consider the code:

  1 01111110 10101010101010101010101

- Sign of mantissa is '-' (leftmost bit is 1)
- Mantissa is 1.10101010101010101010101

  Assumed
- Exponent is -00000010 (=01111110-10000000)

  128
- The number represented is -0.0110101000…000

- Consider the code:

  0 10000001 11001100000000000000000

- Sign of mantissa is '+' (leftmost bit is 0)
- Mantissa is 1.11001100000000000000000

  Assumed

- Exponent is 00000001 (=10000001-10000000)

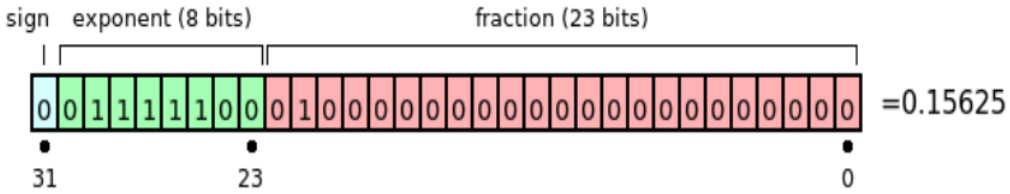
  128

- The number represented is +11.10011000…000

10. IEEE standard 754

1) Single-precision floating point format(单精度浮点格式)

- Almost the same format as we have just described, with some exceptions.
- 32-bits: 1 bit for sign, 8 bits of exponent, 23 of mantissa.
- The exponent is formatted using **excess-127** notation.
- Overall, the standard allows approximately 7 decimal digit precision and approximate value range 10-45 to 1038 .



- The represented number has value v:

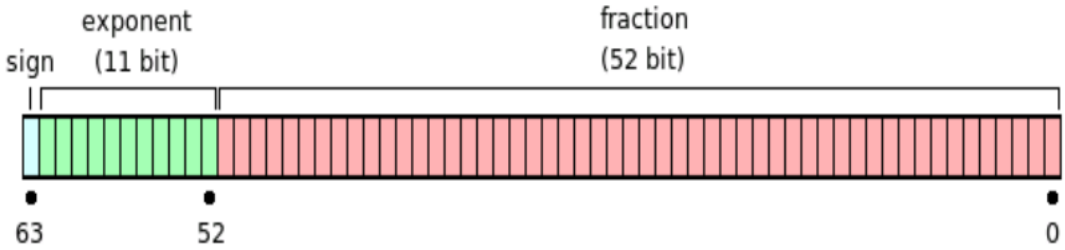
  $v = s \times 2^e \times m$, where

  - $s = +1$ (positive number) when the sign bit is 0;
  - $s = -1$ (negative number) when the sign bit is 1;
  - e = exponent – 127;
  - m = 1.fraction in binary. (The leading '1' is not stored.) Therefore, $1 \le m < 2$.

- In the above example, where $s = 1$, $e = -3$, $m = 1.01$ (in binary, which is 1.25 in decimal).
- The represented number is therefore $+1.01 \times 2^{-3}$ (in binary), which is +0.15625.

19

2) Double-precision floating point format:（双精度）
   - 64-bits: 1 bit for sign, 11 bits of exponent, 52 of mantissa.
   - The exponent is formatted using **excess-1023** notation
   - Overall, the standard allows approximately 15 decimal digit precision and approximate value range 10^{-324} to 10^{308} .



Lecture 19

1. Data storage

1) Storage is the capacity of a device to hold and retain data.(存储是设备容纳和保留数据的容量)
2) Two main types of storage in a computer:
   - Main memory(主存)
   - Mass storage.(大容量存储器)

2. Main memory

1) It refers to physical memory that is internal to the computer
2) The computer can manipulate(操作) only data that is inside the main memory.（计算机只能操作主存中的数据）
3) The amount of main memory in a computer is crucial because:
   - It determines how many programs can be executed at one time.（决定了同时能运行多少程序）
   - How much space can be allocated to a program.（多少空间可以被分配给程序）

3. RAM

1) RAM: Random Access Memory.
   The name is to set it apart from serial tape storage with which you cannot access data stored in the second or following blocks without going through the first block of data.（这个名称是为了将它与串行磁带存储器分开，串行磁带存储器不经过第一个数据块就不能访问存储在第二个或后面的数据块中的数据）
2) The memory can be seen as a set of numbered storage elements, called **words**, each of which contains some information.（存储器可以看作是一组被编号的存储元素，称为 word，每一个 word 都包含一些信息）
3) Each word is numbered with its **address**.（每一个 word 被它的 address 标号）
4) Any word of memory can be accessed “without touching” the preceding words (Random Access).
   （内存中的任何 word 都可以访问，而不需要碰前面的 word(随机访问)。）
5) **Access time** is the same for all the stored items.
   （所有存储项目的访问时间相同）

4. Words, Bytes and Bits

1) A word may be visualised as a set of more elementary storage elements (memory cells), called bits, arranged in a row.（一个 word 可以形象化为一组更基本的存储元素(记忆单元)，称为 bits，排成一行）
2) The number of bits in a word may vary between different computers; 32 bits in a word is common.（不同计算机中 word 的 bits 数不同，一般都是 32bits）

3) A standard unit of 8 bits called a **byte**.

5. Bytes, Kilobytes, Megabytes, etc

- 1 Byte (B) = 8 bits.
- 1 Kilobyte (KB) = 210 bytes = 1024 bytes.
- 1 Megabyte (MB) = 210 KB = 1024 KB.
- 1 Gigabyte (GB)= 210 MB = 1024 MB.
- 1 Terabyte (TB) = 210 GB = 1024 GB

6. Typical sizes（典型大小）

- Typical **RAM** module has 512 MB / 1 GB of memory.
- Common size of **floppy disk（软盘）** is 1.44 MB.
- Common size of **CD** is 650 MB
- Common size of **memory disc（存储盘）** is 1+ GB.
- Common size of **DVD** is 4.7 GB.
- Typical **hard drive（硬盘驱动器）** has the size of 120–200-300 GB.

7. RAM 的两种形式

1) Dynamic RAM (DRAM)（动态 RAM）
   - Cheaper, but slower.
   - Implemented via capacitors（通过电容器实现）
   - DRAM needs to be refreshed.（需要刷新）
2) Static RAM (SRAM)（静态 RAM）
   - Faster, but more expensive.
   - Implemented via flip-flops（通过触发电路实现）
   - No need for refreshing（无需刷新）
3) Both types of RAM are **volatile**（以上两种 RAM 都是挥发性的）
   They lose their contents when the power is turned off.（电源一关啥都没了）

8. Refreshing DRAM（刷新 DRAM）

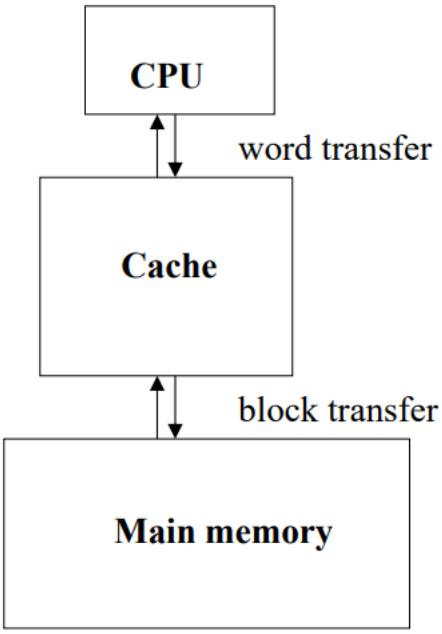
- Real capacitors leak charge.（现实的电容器漏充电）
- Stored data eventually fades.（存储的数据消失）
- To retain data, capacitor charge has to be refreshed periodically.
  （为了保留数据，电容充电必须定期刷新）

9. ROM Chips（控制用只读存储器片）

1) Read-only memory (ROM)
2) Software stored inside also known as firmware（存储在其内部的软件也被称为固件）
3) Helps boot up the system（帮助启动系统）
4) BIOS
   - Basic Input Output System

10. Other Forms of Memory

1) Cache memory ("cash")（缓存）
   - quick access memory, internal or external to the processor（快速存取存储器，内部或外部的处理器）
   - bridge between the processor and RAM（处理器和 RAM 之间的桥梁）
   - including **simultaneous read/write**（包括同步读写）



2) Video memory（图象存储器）

11. Mass storage（大容量存储器）

1) 定义：
   - It refers to various techniques and devices for storing a large amount of data.（它是指用于存储大量数据的各种技术和设备）
   - Unlike main memory, mass storage devices retain data even when the computer is turned off.（与主存储器不同，大容量存储设备即使在计算机关闭时也可以保留数据）

2) Types of mass storage：（类型）
   - Hard disks（硬盘）
   - Optical disks（光盘）: CD-ROM(只读光盘), CD-RW(可擦写光盘), DVD(数字化视频光盘), etc.

12. Hard Disk Drives (HDD)（硬盘驱动器）

1) Hard disk drives are the most important types of permanent storage used in computers (esp. PCs).（硬盘驱动器是计算机(尤其是个人电脑)中最重要的永久存储设备）

2) Hard disks differ from the other mass storage devices in three ways:（硬盘与其他海量存储设备有三个不同之处）
   - Size (usually larger).
   - Speed (usually faster).
   - Permanence (usually fixed in computer and not removable).（永久性(通常固定在计算机中，不可移动)）

13. RAM vs. Mass Storage (HDD)

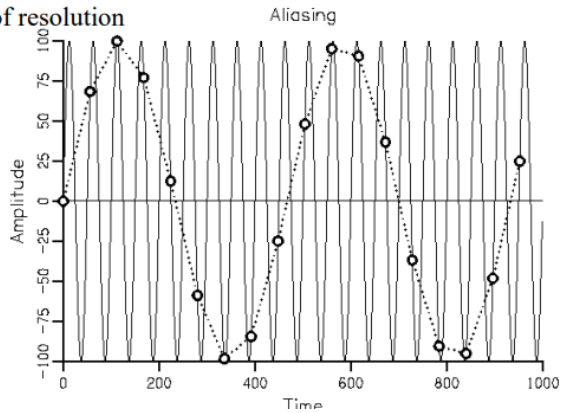
1) RAM is volatile, hard disks are not.（RAM 易挥发，硬盘不是）
2) In general，RAM is much faster than mass storage (hard disks):
   - 10 ns (10-8 s) vs. 10 ms (10-2 s) seek time.
   - It is about one million times faster.
3) But it is more expensive:
   - 50 pounds for 8 GB RAM vs. 50 pounds for 1 TB hard disk（8gb 内存 50 英镑，1tb 硬盘 50 英镑）
   - 1MB of RAM is about 125 times more expensive

14. Storing real-world data in digital（以数字方式存储真实世界的数据）

Analog data → digital data(模拟数据→数字数据)

Converting Analog Signal to Digital: Sampling Rate

- The solid curve represents the analog signal at a comparatively high frequency. Circles show where samples were taken at a relatively low sampling rate. The dotted line illustrates the apparent frequency of the sampled waveform, completing about two cycles in the period that the original signal completed 20 cycles
- Issue of resolution



23

15. Storage Requirements for Digital Audio(数字音频的存储要求)

- CD quality Audio（CD 质量的音频）：
  44.1 KHz sampling rate, 16 bits/sample → 16 bits*44.1KHz = 705,600 bps
- Stereo（立体声）：
  44.1 KHz, 32 bits/sample → 1,411,200bps

Lecture 20

1. Back to memory

1) Any memory location in main memory has its own **address**.(主存中的每个 memory 都有自己的地址)
2) It follows then the more memory the large address are needed.(越多 memory 所需地址就越多)
3) Maximal memory length depends on address width.(最大的 memory 长度取决于地址长度)

2. Address width（地址长度）

Determined by:（取决于）

- the number of the bits in the CPU address registers such as IP, MAR.（CPU 地址寄存器中的位数）
- The number of lines in the address bus.(地址总线中的线数)

3. Address width and memory length

Address width	Maximal memory length
16	64 Kbytes
20	1 Mbytes
24	16 Mbytes
32(Pentium)	4 Gbytes
64(64-bit architectures)	>17 billion Gbytes

4. Memory parameters(内存参数)

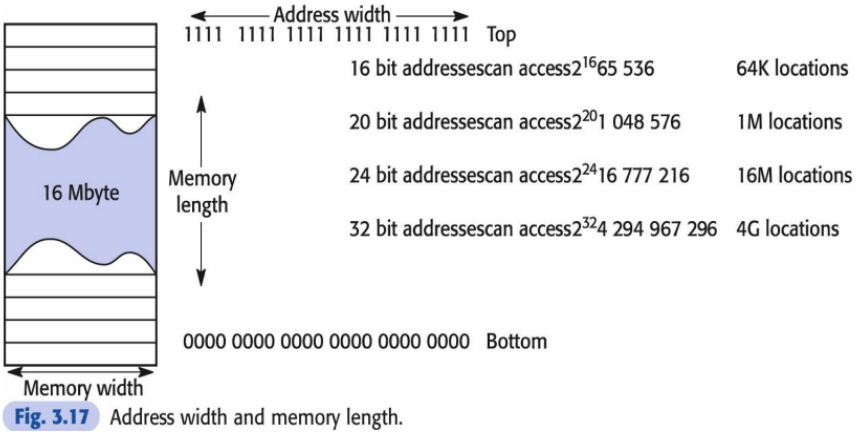
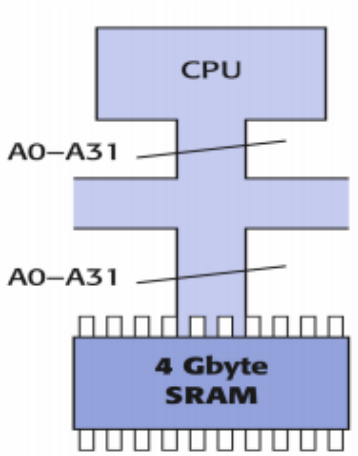


Fig. 3.17 Address width and memory length.

5. Ideal configuration(理想配置)

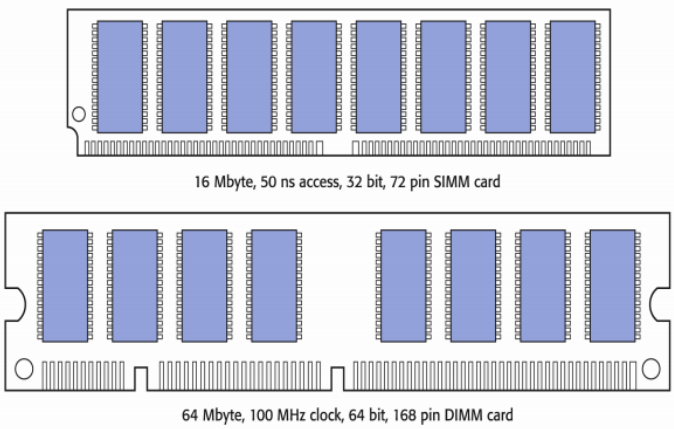
- With 32-bit address width the maximum memory addressing space is 4 Gbytes.
- Ideal configuration would be to have a single memory chip and to send an address via 32 address lines(理想的配置是拥有单个内存芯片，并通过 32 条地址行发送地址)



6. Not so ideal in practice

- 4Gbyte is still a lot of space and many systems do not use it all.(4Gbyte 仍然是大量的空间，许多系统不使用它的所有)
- Memory chips (DRAM) have a typical size 128 Mbits – 256 Mbits – 1 Gbits and they are combined into the memory modules of 8 – 16 chips.
  (内存芯片（DRAM）的典型尺寸为 128 Mbits-256 Mbits-1 Gbits，它们组合到 8 + 16 芯片的内存模块中)
- For example, the module of 8 of 256 Mbit chips has a capacity of 8 * 256 Mbits = 256 Mbytes.
  (例如，256 Mbit 芯片中的 8 个模块的容量为 8 * 256 Mbit = 256 MB)
- To address memory, we need to select memory chips as well as locations within the chips.(为了寻址内存，我们需要选择内存芯片以及芯片内的位置)

7. Memory modules(内存模块)



8. Memory mapping

1) When the CPU sends out an address:
   - A part of the address locates the correct chip(地址的一部分定位正确的芯片)
   - Another part specifies an address within the correct chip.(另一部分在正确的芯片中指定一个地址)
   - How actually the addresses are mapped to the memory locations is defined by memory maps.(地址实际上是如何映射到内存位置的是由内存映射定义的)
2) Memory map for a small system

Device	Size	Pins	32 bit address bus	Address range
PROM1	1 Mbyte	20	0000 0000 xxxx ++++ ++++ ++++ ++++ ++++	0000 0000 – 000F FFFF
RAM1	16 Mbyte	24	0000 0001 ++++ ++++ ++++ ++++ ++++ ++++	0100 0000 – 01FF FFFF
RAM2	16 Mbyte	24	0000 0010 ++++ ++++ ++++ ++++ ++++ ++++	0200 0000 – 02FF FFFF
RAM3	16 Mbyte	24	0000 0011 ++++ ++++ ++++ ++++ ++++ ++++	0300 0000 – 03FF FFFF
RAM4	16 Mbyte	24	0000 0100 ++++ ++++ ++++ ++++ ++++ ++++	0400 0000 – 04FF FFFF

\+ address line used directly for internal selection
x line ignored, indicates partial (degenerate) addressing
0 must be 0 for chip selection
1 must be 1 for chip selection

Fig. 6.13 Memory map for a small computer system.

9. Memory address decoding(内存地址解码)

1) Memory chips are not normally matched to the width of the address bus. For example(内存芯片通常不匹配地址总线的宽度)
   - CPU may send 32-bit address.
   - RAM may receive directly 24-bit address.
2) Special **Memory Address Decoding** circuit implements necessary decoding (特殊存储器地址解码电路实现必要的解码)

10. Why we need memory levels?

- To keep up with the demands of faster CPUs.（以满足更快的 cpu 的需求）
- To limit system cost.（限制系统成本）
- To cope with ever-expanding software systems.（以应付不断扩展的软件系统）

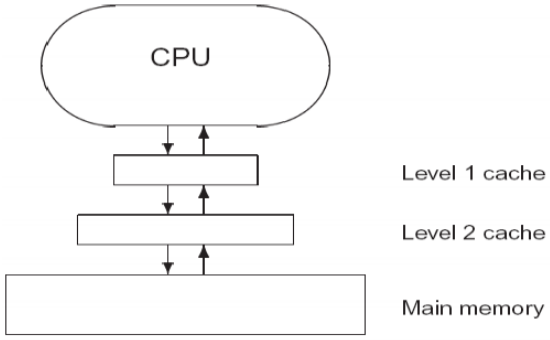
11. Registers

- Registers are the memory cells which are core part of the processor itself.（寄存器是处理器本身的核心部分的内存单元。）
- It has very fast access (a few nanoseconds).（它的访问速度非常快(几纳秒)）
- Not that much memory: tens of 32-, 64-, 80- bit registers (typically).（内存不多：32、64、80 位寄存器（通常））

12. Cache memory

- A memory (more expensive, but faster SRAM) placed between CPU and main memory.（放置在 CPU 和主存之间的一种内存(更贵，但更快的 SRAM)）
- Contains a copy of the portion of main memory.（包含主存部分的副本）
- The aim is to maintain in fast cache the currently active sections of code and data.（目的是将当前活动的代码和数据段保持在快速缓存中）
- Processor when needs some information first checks cache.（处理器在需要一些信息时首先检查缓存）
- If not found in cache, the block of memory containing the needed information is moved into the cache.

13. Levels of cache



Typically Level 1 cache has the size 8-64 KB.
Typically Level 2 cache has the size 128-512KB

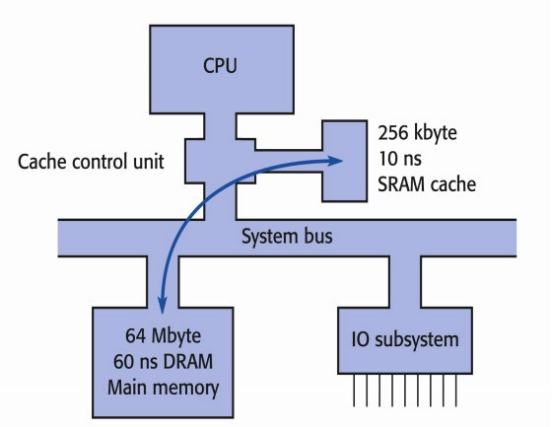
14. Localisation of access

1) The idea of cache memory exploits(利用) **Localisation of Memory Access** principle:
   - Computers tend to spend periods of time accessing the same locality of memory.(计算机倾向于花费一段时间来访问相同的内存位置)
2) Therefore:
   - A portion(部分) of code or data which require access needs to be loaded into the fastest memory nearest to CPU.(需要访问的部分代码或数据需要加载到最接近 CPU 的最快内存中)
   - Other sections of the program and data can be held in readiness lower down the memory hierarchy.(程序和数据的其他部分可以保存在内存层次结构的较低的就绪状态中)

15. Why localisation of access works?

- Partly due to the programmer clustering related data items together in arrays or records.(部分原因是程序员将相关数据项聚集在数组或记录中)
- Partly due to the repeating patterns(模式) in a program (i.e. loops)
- Partly due to the compiler attempting to organise the code in an efficient manner(部分原因是编译器试图以一种有效的方式组织代码)

16. Cache memory and cache control unit



17. Memory hierarchy(分级存储器体系)

Going down the hierarchy(向下层次结构)

- Increased capacity(容量增加)
- Increased access time(增加访问时间)
- Decreased frequency of access of the memory by the processor.(降低了处理器访问存储器的频率)
- Decreased cost per bit.(降低每比特成本)

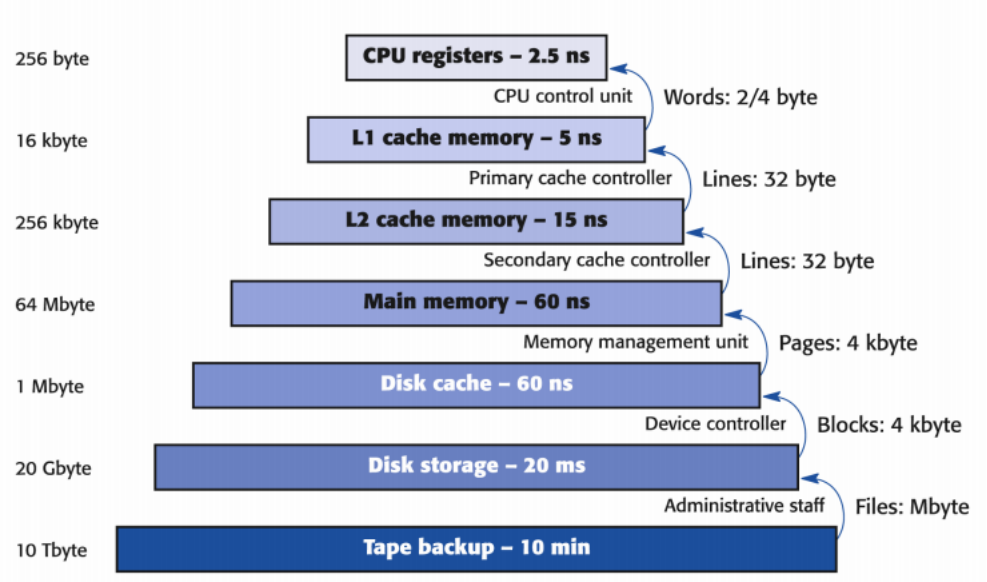
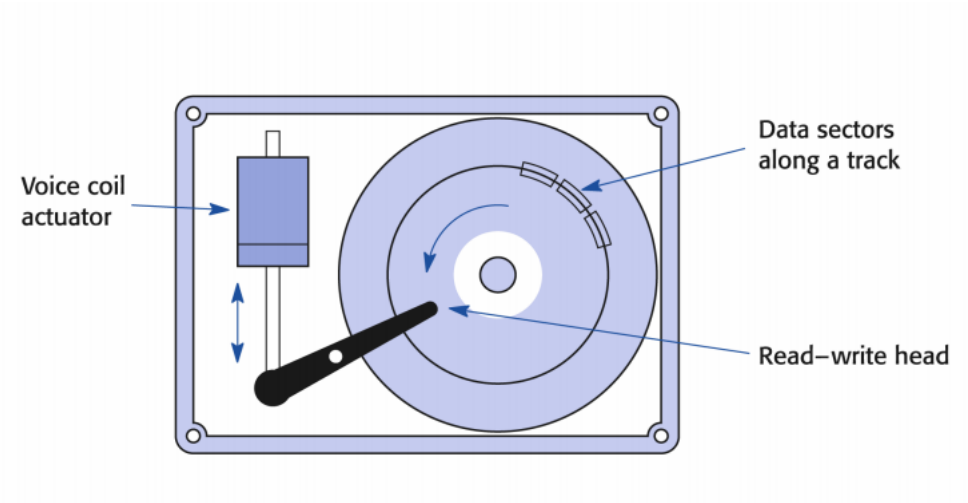


Fig. 12.2 Memory performance and storage hierarchy.

Lecture 21

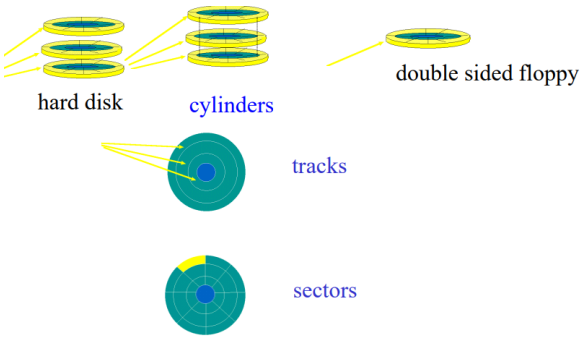
1. Hard Disk Drives (HDD)

1) Hard disk drives are the most important type of permanent storage used in computers (esp. PCs).
2) Hard disks differs from the other mass storage devices in three ways:（与其他大容量存储器的不同有三点）
   - Size (usually larger).
   - Speed (usually faster).
   - Permanence (永久性)(usually fixed in computer and not removable).
3) Schematic diagram of hard disk(硬盘示意图)



voice coil actuator: 音圈致动器
data sectors along a track: 轨道上的数据扇区
read-write head: 读写头

4) Tracks, sectors and cylinders(轨道，扇区和圆柱体)
   - Each disk platter has its information recorded on both **surfaces**.
   - Each platter has two **heads**.（每个磁盘有两面）
   - The information is recorded in concentric circles called **tracks（磁道）**.(信息记录在称为磁道的同心圆中)
   - Each track(磁道) is broken down into smaller pieces called **sectors（扇区）**, each of which holds 512 bytes of information.(每个磁道被分解成更小的部分，称为扇区，每个扇区包含 512 字节的信息)



➢

5) Addressing（地址）
   - ➢ CHS.
     – Cylinder, Head, Sector system.
     – Telling the disk controller which cylinder, head and sector to access.
     – Can be mapped onto LBA.
   - ➢ LBA
     – Large Block Addressing, by absolute number of a sector.（大块寻址，按扇区的绝对数量表示。）

从磁盘中搜索数据的过程

- **Example (simplified):**

A piece of information needs to be read.

1. The first step is to figure out where on the disk to look for the needed information.
2. The location on the disk → address expressed either:
   - In terms of the **cylinder, head and sector** (CHS).
   - Or, in terms of the **absolute sector number** (LBA).
3. A request is sent to the drive over the **disk drive interface** giving it this address and asking for the sector to be read.

2. Progress for the last 25 years

- ➢ First PC hard disks: capacity 10 MB and cost about 100 pounds (RMB$1100) per 1 MB.
- ➢ Now: capacity > 300 GB and cost < half penny for 1 MB.

3. Hard disk vs. main memory

- ➢ It is larger.
- ➢ It is slower.
- ➢ It is cheaper (per 1MB)

4. Disk cache

1) A portion of main memory used as a buffer(缓冲器) to temporarily(暂时的) hold data for the disk.
2) Disks write operations are **clustered**.(磁盘的 write 是集中的)
3) Some data written out may be needed again.（一些数据可能需要再次写入）
   --- The data are retrieved rapidly from the **disk cache** instead of slowly from **disk**.（从**磁盘缓存**中检索数据的速度很快，而从**磁盘**中检索数据的速度很慢）
4) Java "*flush()*" in *java.io.PrintWriter*.

5. Storage Technology（存储技术）

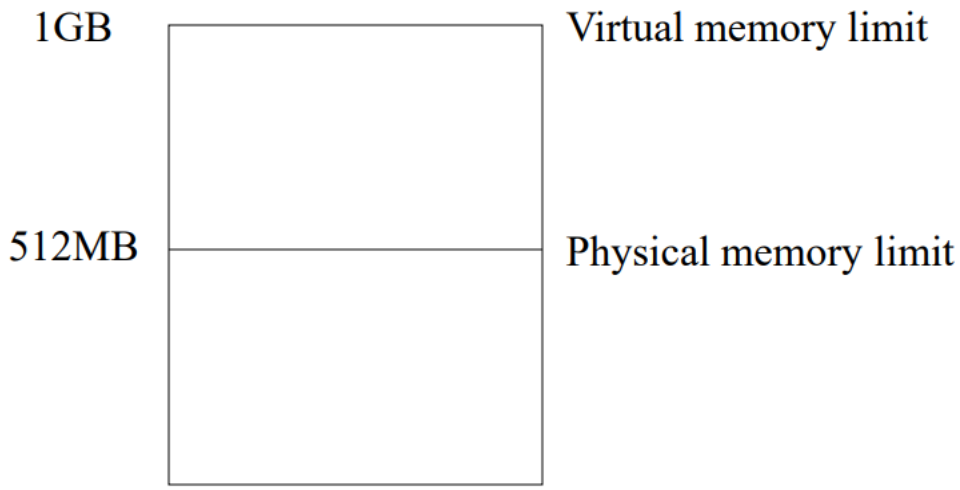
1) Retrieving（检索） files into RAM is called **reading**
   - loading an application
   - opening a file
   - files can be programs or documents
2) Copying data *from RAM onto a secondary storage* device is called **writing.**

6. Files, records, fields, keys

1) Files: e.g. PERSONNEL FILE
2) Records:
   Example: Adam's personal data:
   Adam Smith 35 Manager Purchasing
3) Fields: e.g. name, age, position, job function
4) Key: e.g. Adam Smith

7. Virtual memory(虚拟内存)

1) **Virtual memory** is a technique, in a sense, opposite to **caching**: (从某种意义上说，虚拟内存是一种与缓存相反的技术)
   - It is the use of low-level memory (i.e. hard disk) to 'expand' high-level (main) memory.(它是使用低级内存(即硬盘)来扩展高级(主)内存)
   - It provides a convenient expansion of main memory by 'overflowing' data and program code onto magnetic disk.(通过将数据和程序代码溢出到磁盘上，它提供了方便的主存扩展)
2) The area **on disk** reserved for this purpose is known as the **swap area.(磁盘上为此保留的区域称为交换区域)**

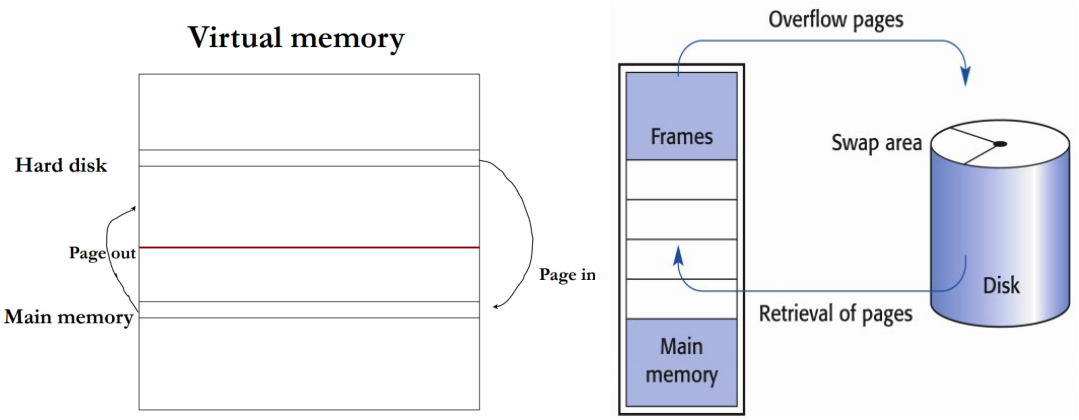


8. Memory Management

1) Virtual memory
   - hard disk space(硬盘空间)
   - when processor needs more RAM space, swaps unused data onto designated hard disk space.(当程序需要更多的 RAM 空间时，把无用数据扔到硬盘上)
   - improves flexibility but is slower than RAM to which the processor has direct access（增强了灵活性到那时比起直接访问 RAM 要慢）

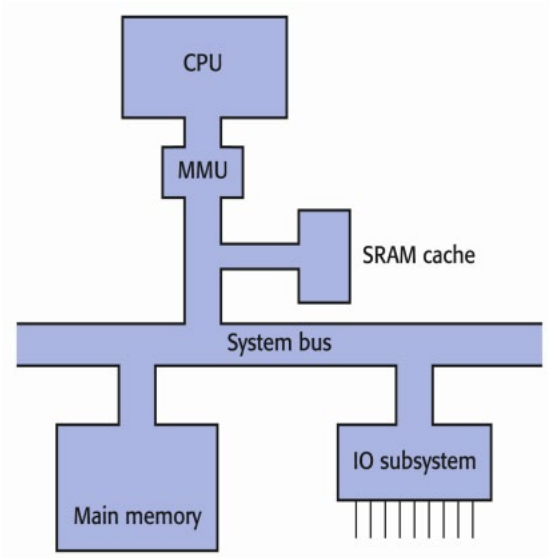
9. Virtual Memory Management（虚拟内存管理）

1) Main memory is divided into frames（帧）, often 4KB.
2) The executable program is similarly divided into frame-sized chunks known as pages.（可执行程序以类似的方式划分为称为页的帧大小的块）
3) When a program is invoked（被召唤） not all the pages are loaded into main memory, only sufficient to get it started. (只要把足够启动它的东西扔进主存就 OK 了)
4) The rest are copied into the disk area, known as swap area.（剩下没资格进入主存的就被打入硬盘的 swap area）
5) When an instruction is needed from a page not yet in the main memory it is loaded from the disk.（当一条指令不在主存中，就从硬盘中 load）
6) If no empty frames exist at the moment *the least used frame* is freed to allow the new pages to be loaded. (如果当下没有空的 frame, 则将最少使用的 frame 内数据覆盖掉)
   --- This is called **swapping**



10. Virtual Memory Addressing（虚拟内存的地址）

1) Within an user program addresses are in a form of 32 bit **logical** address.
2) In the case of 4KB paging system:
   - The lower 12 bits are '**address within a page**'.
   - The upper 20 bits serve as the '**page number**'
3) **Memory Management Unit（内存管理单元）** maps logical addresses into references to frame numbers and addresses within the frames.（**内存管理单元**将逻辑地址映射到对帧内帧编号和地址的引用。）



Lecture 22

1. Building computers from logic

1) Computer systems may be described at different levels of understanding.
2) At the **architectural** level the computer is described as a machine for executing instructions.(从构造的角度来说计算机时一个用来运行指令的机器)
   - This level is most appropriate for understanding how programs are executed

2. Engineering level（工程层面）

1) Engineering model of the computer represents the machine as a complex electrical circuit.（计算机的工程模型将其表现为一个复杂的电路）
2) Within the circuit there are a large number of physical connections, along each of which a current may flow during the operation of the machine.
   - Presence of a current is used to represent transmission of the binary digit 1, while
   - Absence of a current represents a value 0.

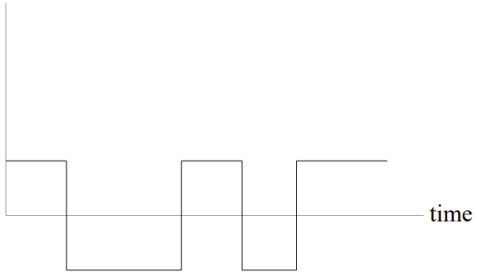
3. Digital Systems（数字系统）

Digital systems based on *electronic circuitry*

- 1s and 0s, or on and off
- Each 1 or 0 is called a bit; or binary digit
- Computers use digital data representation（计算机使用数字数据来表示）

4. Digital Signal（数字信号）

1001011…



5. Digital electronic circuit（数字电路）

- This kind of circuit is called **digital electronic circuit**, because the relevant characteristic is the presence（出现） or absence（消失） of current (digit 1 or 0), rather than the amount of current flowing.

6. Analog Systems（模拟系统）

Analog

1) continuously variable values, along a range, such as（连续可变值，沿一个范围）temperature and pressure values（压强）
2) traditional analog recording devices（传统的模拟信号记录） are
   - humidity recorders（湿度记录）
   - mercury thermometers（水银温度计）
   - pressure gauges（压力表）
3) standard telephone lines transmit analog signals（标准电话线传输模拟信号）

7. Boolean Operations and Boolean Gates（布尔运算和布尔门）

- All operations that computers perform may be defined in terms of **basic boolean functions**, **operating on bits**.
- The digital electronic circuits and their components can be built from the devices implementing basic boolean operations – boolean gates (logic gates).（数字电子电路及其组件可以从实现基本布尔操作的设备（布尔门（逻辑门）中构建）

8. Boolean Gates（布尔门）

1) Consider the circuit device called **AND gate**:



2) Two of these, labelled A and B, represents electronic **inputs** to the component, along each of which a current may
   - flow (boolean value 1) or may
   - not flow (boolean value 0).
3) Third connection, labelled Y represents the **output** of the component

9. AND gate

1) In the case of AND gate current will flow in Y if and only if a current flows in the input A **and** in the input B.（A&B 都有电流，Y 才有电流）



2) One may write Y = A **and** B.
3) Thus, this component implements logic (boolean) operation AND.（因此这个组件实现了布尔运算“**AND**”）

10. Truth tables for basic logic operations（基本逻辑运算的真值表）

A B	A **and** B
0 0	0
0 1	0
1 0	0
1 1	1

A B	A **or** B
0 0	0
0 1	1
1 0	1
1 1	1

A	**not** A
0	1
1	0

A B	A **xor** B
0 0	0
0 1	1
1 0	1
1 1	0

A B	A = B
0 0	1
0 1	0
1 0	0
1 1	1

11. Alternative notations（替代符号）

- and: ∧, &
- or: ∨
- xor: |
- not: ¬, -

12. Other standard logic gate drawing representations:



OR gate



NOT gate



XOR gate

13. Basic logic gates and their truth tables

Inputs A B	C A AND B
0 0	0
0 1	0
1 0	0
1 1	1

Inputs A B	A OR B
0 0	0
0 1	1
1 0	1
1 1	1

Inputs A B	A XOR B
0 0	0
0 1	1
1 0	1
1 1	0

Inputs A	NOT A
0	1
1	0

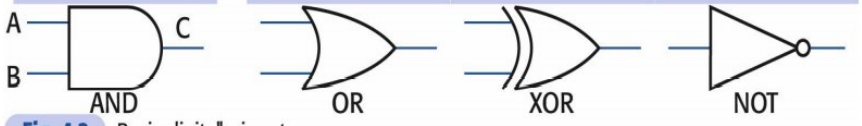


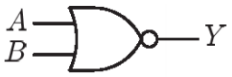
Fig. 4.2 Basic digitallogic gates.

© Pearson Education 2001

14. More boolean gates

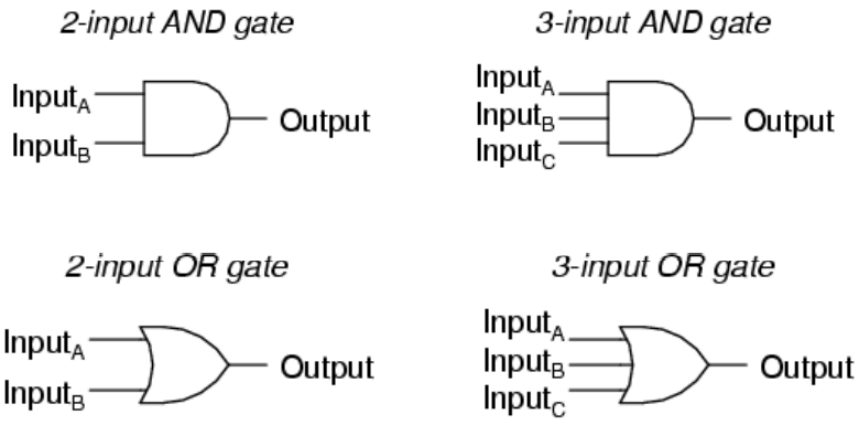


NAND gate, $Y =$ **not** $(A$ **and** $B)$



NOR gate, $Y =$ **not** $(A$ **or** $B)$

15. Three-input gates



16. Boolean circuits

- Elementary boolean gates can be combined into boolean circuits, implementing more complex boolean functions (operations).（基本布尔门可以组成布尔电路，来实现更加复杂的布尔运算）
- In fact, any boolean function can be implemented with this set of basic boolean gates.（事实上任何的布尔函数都可以同通过这组基本的布尔门实现）

17. Data flow control circuit – Filter（数据流控制电路-滤波器）

D X	Out
0 0	0
0 1	0
1 0	0
1 1	1

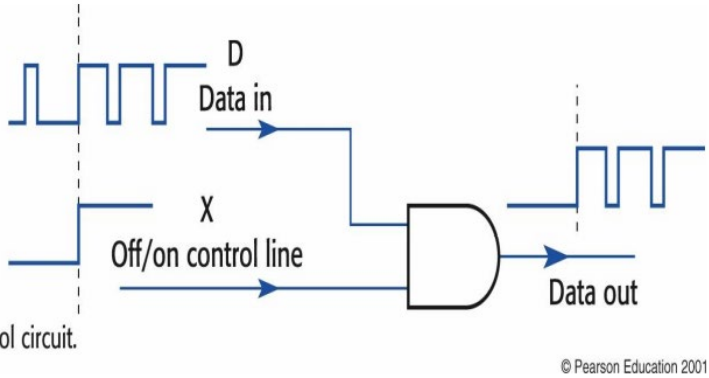
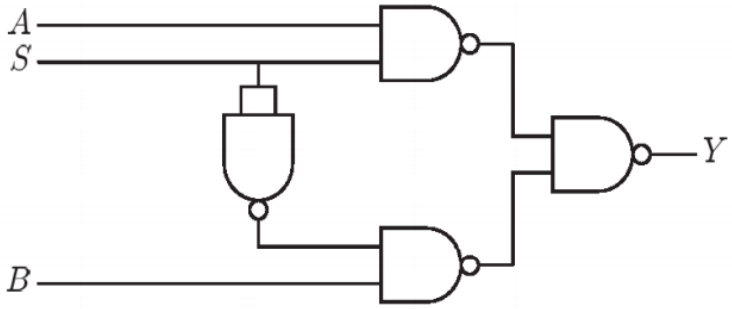


Fig. 4.4 Data flow control circuit.

© Pearson Education 2001

18. Selector circuit（选择电路）

Selector circuit

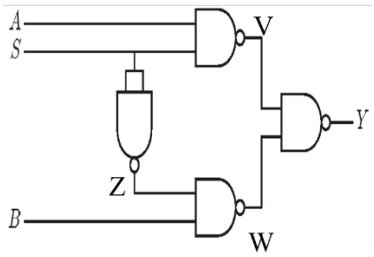


If S = 1 then Y = A

if S = 0 then Y = B

Lecture 23

1. Selector circuit



1) This circuit implements the function:
   - *Y* = **not** (*V* **and** *W*),
     where,
   - *V* = **not** (*A* **and** *S*)
   - *W*= **not** (*Z* **and** *B*)
   - Z = **not** (*S* **and** *S*) = **not** *S*
2) Combining altogether we get:
   Y = **not** (**not** (*A* **and** *S*) **and not** (**not** (*S*) **and** *B*))
3) **Truth table for selector circuit**

A B S	A and S	V	Z	Z and B	W	V and W	Y
0 0 0	0	1	1	0	1	1	0
0 0 1	0	1	0	0	1	1	0
0 1 0	0	1	1	1	0	0	1
0 1 1	0	1	0	0	1	1	0
1 0 0	0	1	1	0	1	1	0
1 0 1	1	0	0	0	1	0	1
1 1 0	0	1	1	1	0	0	1
1 1 1	1	0	0	0	1	0	1

2. Selector function

One may give a short definition of the function from the truth table:

If S = 1 then Y = A

if S = 0 then Y = B

Or **Y = (S&A)V(-S&B)**

3. Data selector, or multiplexer(数据选择器或多路复用器)

Control W X Y Z	Data A B C D	OUT
0 0 0 1	a b c d	a
0 0 1 0	a b c d	b
0 1 0 0	a b c d	c
1 0 0 0	a b c d	d

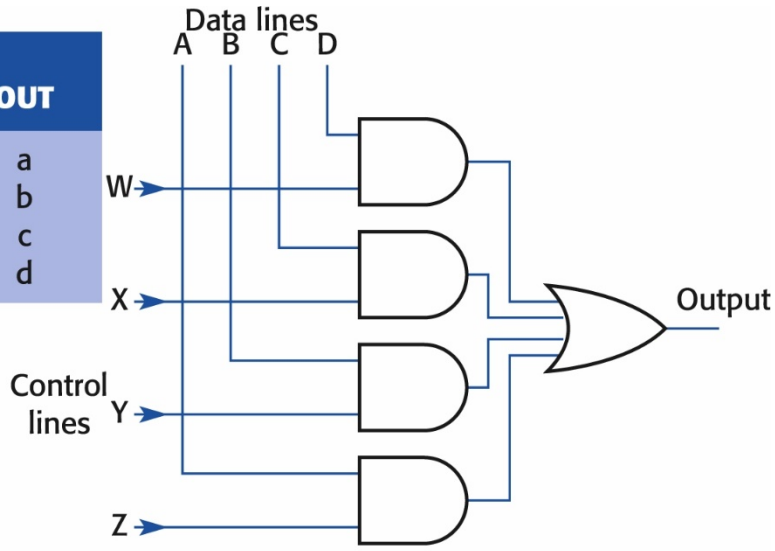


Fig. 4.5 Data selector circuit.

© Pearson Education 2001

4. How to design the circuit?

- The circuit upper is a straightforward implementation of the function(上面的电路是一个简单的功能实现

$$O = (A \text{ and } Z) \text{ or } (B \text{ and } Y) \text{ or } (C \text{ and } X) \text{ or } (D \text{ and } W).$$

5. The problem with the circuit

- What happens if more than one control line has the signal 1?
  --- The circuit (the function it implements) does not behave as a selector
- Can we do better?
  Yes, use **two-line decoder** to replace four control lines(是的，用两线解码器代替四条控制线)

6. Two-line decoder(两线解码器)

Selector Y X	Line d c b a
0 0	0 0 0 1
0 1	0 0 1 0
1 0	0 1 0 0
1 1	1 0 0 0

7. Implementation of two-line decoder(两线解码器的实现)

- The implementation follows the description: （该实现遵循以下说明）
  - Detect(检测) pattern 00 (on YX lines), output the result from line A.
  - Detect pattern(模式) 01, output the result from line B.
  - Detect pattern 10, output the result from line C.
  - Detect pattern 11, output the result from line D

8. Data selector with two-line decoder

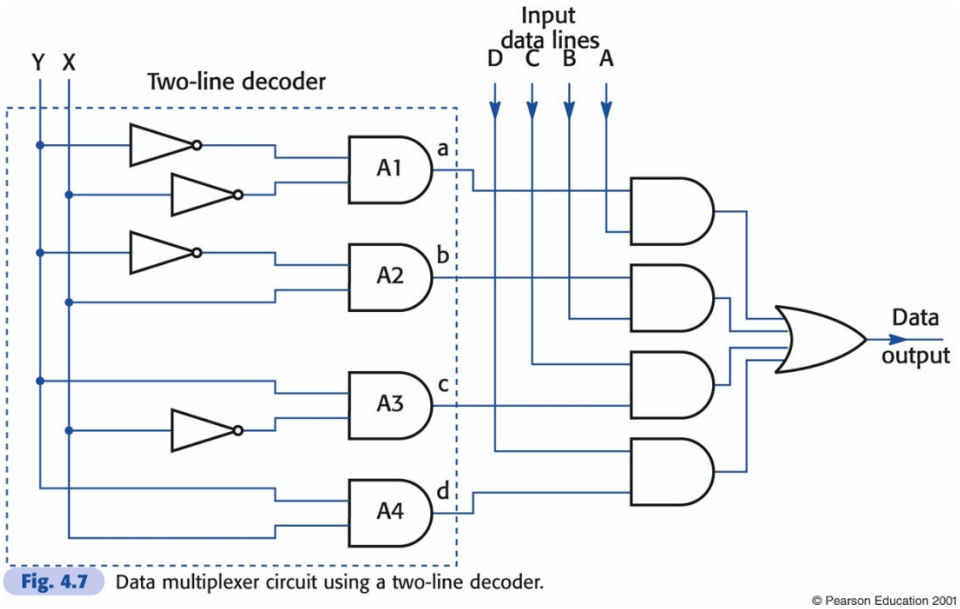


Fig. 4.7 Data multiplexer circuit using a two-line decoder.

© Pearson Education 2001

9. Data selector with two-line decoder: logic equation

$$O = (A \textbf{ and } (\textbf{not } X \textbf{ and not } Y))$$
$$\textbf{or}$$
$$(B \textbf{ and } (X \textbf{ and not } Y))$$
$$\textbf{or}$$
$$(C \textbf{ and } (\textbf{not } X \textbf{ and } Y))$$
$$\textbf{or}$$
$$(D \textbf{ and } (X \textbf{ and } Y)).$$

10. Cost comparison in gate count

- Multiplexer（多路(复用)器）： 4 AND gates + 1 OR gate
- 2-line decoder（两线解码器）： 8 AND gates + 1 OR gate + 4 NOT gates

11. Implementing a function（实现一个 function）

Given a truth table for a logic function, to implement the function by a logic circuit one may proceed as follows:（为逻辑 function 给出 truth table）

- Implement detectors (i.e. using AND/NOT gates) for all input patterns on which the function gives the output 1.（为函数给出输出的所有输入模式实现检测器(即使用 AND/NOT 门)）
- Connect the outputs of all detectors to the inputs by an OR gate.（通过 OR 门将所有检测器的输出连接到输入端）

12. Example：

Truth table

i_1	i_2	i_3	i_4	O_1
1	1	0	0	1
1	1	0	1	1
0	0	1	0	1
1	0	1	0	1
0	1	0	1	1
.....				0
…				0
.....				0

The function gives 0 on all other inputs.

Short form

i_1	i_2	i_3	i_4	O_1
1	1	0	*	1
*	0	1	0	1
0	1	0	1	1

- Logic expression:

$$O_1 = (i_1 \textbf{ and } i_2 \textbf{ and } (\textbf{not } i_3)) \textbf{ or } ((\textbf{not } i_2) \textbf{ and } i_3 \textbf{ and } (\textbf{not } i_4)) \textbf{ or } ((\textbf{not } i_1) \textbf{ and } i_2 \textbf{ and } (\textbf{not } i_3) \textbf{ and } i_4).$$

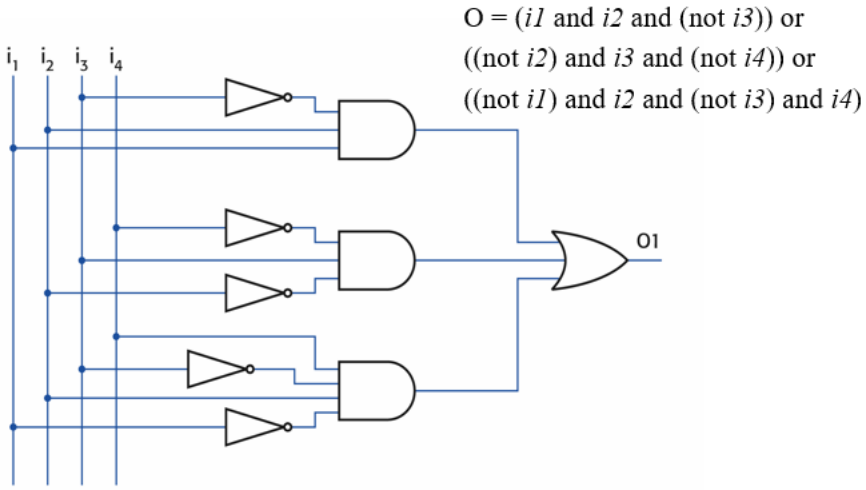


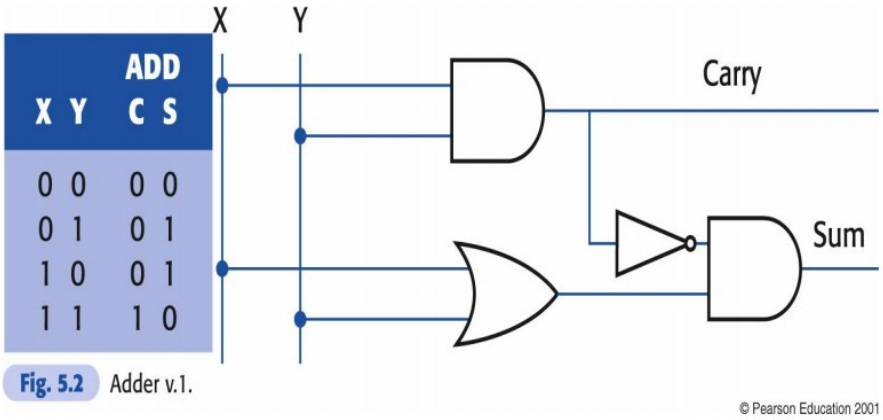
Fig. 4.9 Implementing a sum-of-products logic equation.

（Sum-of-products Implementation）

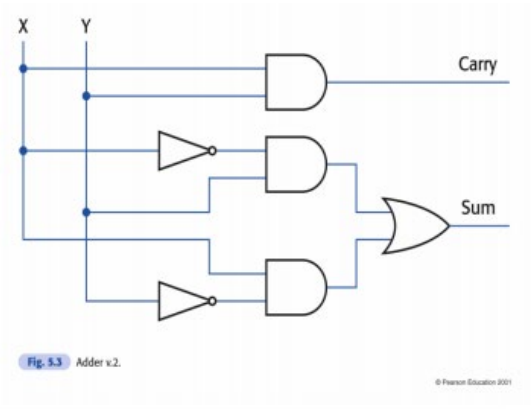
Lecture 24

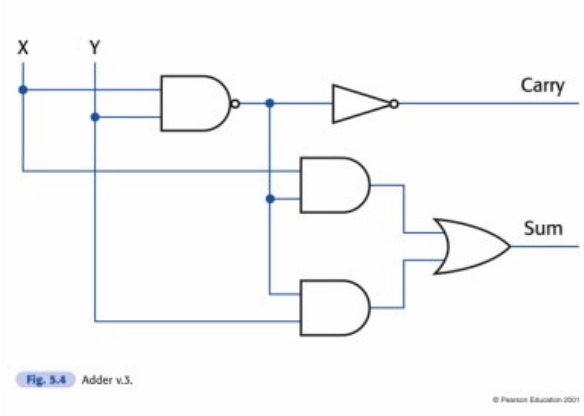
1. Doing arithmetic via logic（通过逻辑计算）

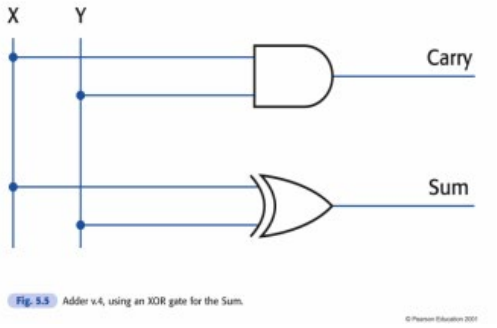
- Binary addition. For the addition of single-bit binary numbers: (Half-)adder
  （二进制加法。用于单位二进制数的加法:(半)加法器）



- Other possible designs for half adder







2. Full adder（全加器）

- For the addition of multi-bit binary numbers one needs to deal with carry-in from previous stage, that means the adder should have three inputs.（对于多位二进制数的加法，需要处理前一阶段的进位运算，这意味着加法器应该有三个输入）

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

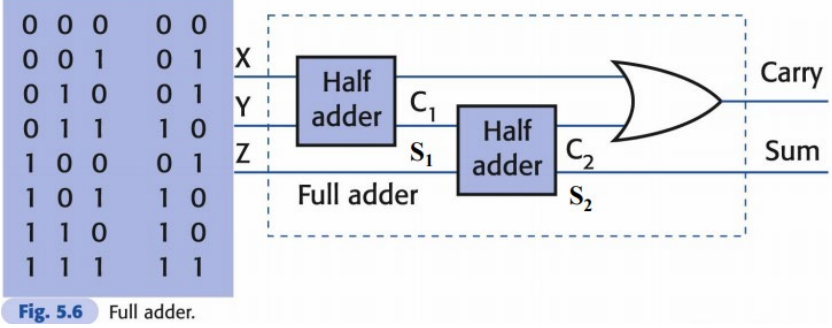


Fig. 5.6 Full adder.

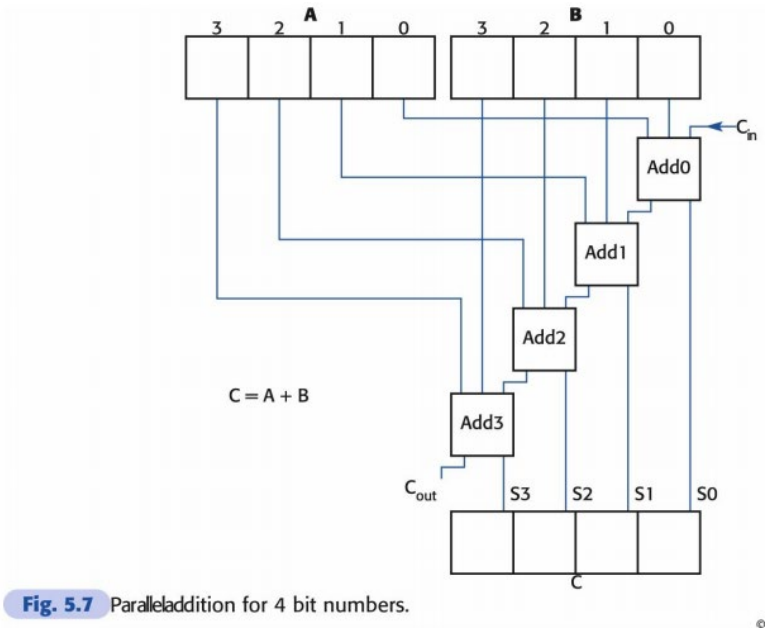


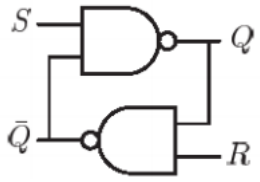
Fig. 5.7 Paralleladdition for 4 bit numbers.

3. Sequential logic circuits（时序逻辑电路）

1) Combinational (combinatorial) logic circuits（组合逻辑电路）
   - In all Boolean circuits that we have seen so far, the output at any moment depends only on the input at the same moment.（在我们目前看到的所有布尔电路中，任何时刻的输出只依赖于同一时刻的输入）
   - **No memory** is there.（还没有 memory 出现☹）
2) Sequential logic circuits（时序逻辑电路）
   - The output depends also on the state of the circuit. The state of the circuit is somehow stored within the circuit.（输出也取决于电路的状态。电路的状态以某种方式存储在电路中）
   - There is a memory inside.（这憨批玩意有 memory 了啊啊啊）

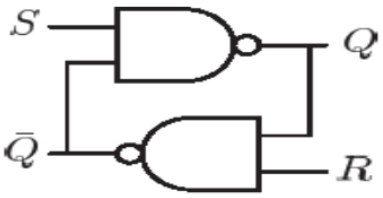
4. Flip-flops（触发电路）, or latches（门闩线路）

- The basic memory element of the sequential circuits is called a flip-flop, or latch.（时序电路的基本存储元件称为触发器或锁存器）
- The simplest flip-flop is made up of two NAND gates. It is called set-reset (SR) flip-flop.（最简单的触发器由两个 NAND 门。它被称为设置重置 （SR） 触发器）



- Inputs are and while outputs are and

5. SR flip-flop



- Suppose that S and R are both initially set to 1. Can we determine two outputs Q and $\bar{Q}$? **No.** Two variants are possible:

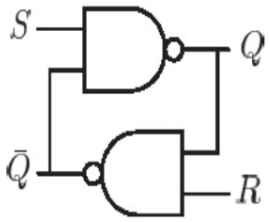
$Q = 1$ and $\bar{Q} = 0$, or

$Q = 0$ and $\bar{Q} = 1$

- Everything is consistent with any of these variants.

- If we fix both inputs to be 1, then the circuit may have one of the two possible states. The circuit is stable as long as R and S remain at 1.（如果我们把两个输入都固定为 1，那么电路可能有两种可能的状态之一。只要 R 和 S 保持在 1，电路就稳定。）
- Let's assume that flip-flop is in a stable state with Q=1
- Suppose, R momentarily becomes 0. This forces changing the value of Q to 0. The circuit will stay in this state even after input R returns to 1. The flip-flop **switched** the states.（突然的把一个输入端的值变成 0，整个系统仍然维持原态，等待被变的端恢复 1）
- If then input S momentarily becomes 0 the system will switch the state back.

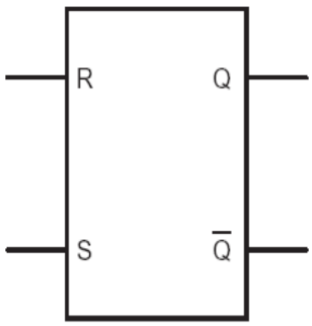
- Thus, SR flip-flop **remembers** which input was set (momentarily) to 0 last.
- One constraint: the logic surrounding this flip-flop must avoid situations where **both** R and S are 0 at the same time.



6. State table of SR flip-flop

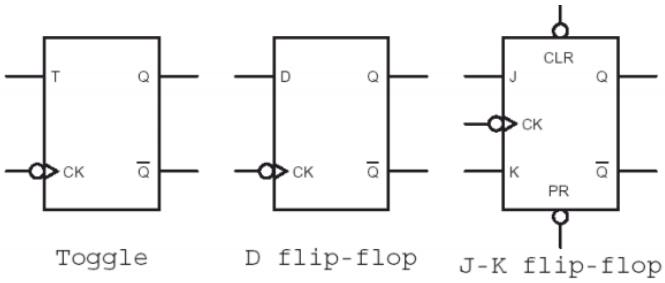
R	S	Q
0	0	?
0	1	0
1	0	1
1	1	Q_{prev}

7. SR representation



FlipFlop(SR)

8. Other types of flip-flops



Toggle D flip-flop J-K flip-flop

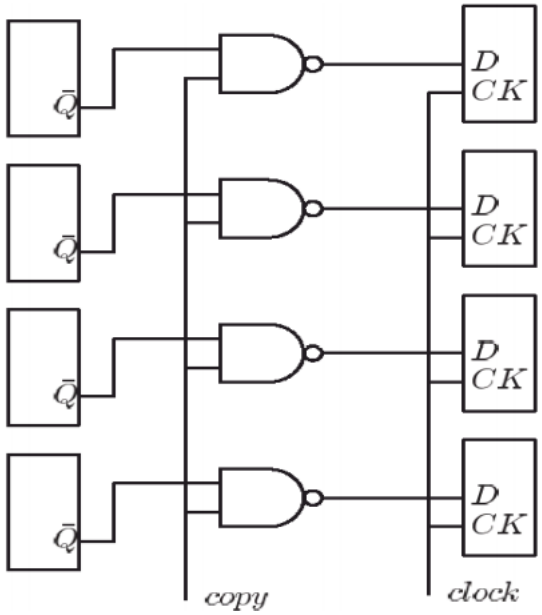
T | Q
0 | Q_{prev}
1 | $\bar{Q}_{prev}$

D | Q
0 | 0
1 | 1

J K | Q
0 0 | Q_{prev}
0 1 | 0
1 0 | 1
1 1 | $\bar{Q}_{prev}$

- All these flip-flops has an input marked for clock. The new output occurs when the clock is pulsed, i.e. momentarily changed from 1 to 0.（所有这些触发器都有一个标记为时钟的输入。当时钟处于脉冲状态时，即从 1 瞬间变为 0，就会产生新的输出）
- CLR (clear) and PR (preset) inputs are used to initialise the flip-flop to known value (0 and 1, respectively).（CLR(清除)和 PR(预设)输入用于将触发器初始化到已知值(分别为 0 和 1)。）

9. Use of D flip-flops - Copying data



Circuit to copy data from one register to another.